Bangladesh University of Engineering and Technology

Department of Electrical and Electronic Engineering

Course No: EEE 310

Project Report on

Error Correction in 15 bits Digital Data Transmission by Hamming Code

Group No: 05, Section: B2

Moshiur Rahman	2006099
Shahriar Rashid Khan Sifat	2006117
Tanvir Arafat Fahim	2006118
Md. Sazidur Rahman	2006121

Instructors:

1. Azazul Islam
Lecturer, Dept. of EEE, BUET

2. Nawshad Binta Nizam
Lecturer, Dept. of EEE, BUET

Table of contents:

SL no.	Description	Page no.
1.	Abstract	3
2.	Introduction	3
3.	Relevant theories	4
	Redundancy for Error Correction	4
	(15,11,4) Hamming Code System	5
4.	Flow Diagram	12
5.	Design	13
	Schematic Diagram	13
	Simulation Output	20
6.	PCB Design	24
7.	Hardware Implementation	27
8.	Testing the circuit	28
	Test 1	28
	Test 2	31
	Test 3	34
9.	Reflection on Teamwork	37
10.	Individual Contribution of Each Member	37
11.	Time-line of Project Implementation	38
12.	Cost Analysis	39
13.	Cash Memos	40
14.	Future Works	43
15.	Applications	43
16.	Summary	43
17.	Conclusion	43
18.	References	44

Abstract:

This report investigates the application of Hamming Code for error correction in the context of 15-bit digital data transmission. The project focuses on implementing the (15,11) Hamming Code, integrating four parity bits to enhance error detection and correction capabilities. Through theoretical exploration and practical implementation, the study assesses the efficiency and reliability of the Hamming Code in maintaining data integrity in the face of single and multiple-bit errors. The findings contribute to a deeper understanding of error correction techniques in digital communication systems, particularly within the specific constraints of 15-bit data sets.

Introduction:

In the realm of digital data transmission, ensuring the integrity and accuracy of transmitted information is paramount. Errors can occur due to various factors such as electromagnetic interference, noise, or hardware malfunctions. To address this challenge, error correction codes play a pivotal role in detecting and rectifying errors in transmitted data. The Hamming Code, named after its creator Richard Hamming, is a widely used error correction technique that adds redundancy to the transmitted data, allowing for the identification and correction of errors. Here, the basic idea is to add parity bits to the original data bits to create a code that can detect and correct single-bit errors in data. This project centers around the implementation and exploration of the (15,11,4) Hamming Code system. The (15,11,4) Hamming Code, characterized by its incorporation of four parity bits, possesses the capability to detect and correct single-bit errors while identifying multiple-bit errors. This feature establishes it as an efficient and dependable solution in the realm of digital data transmission.

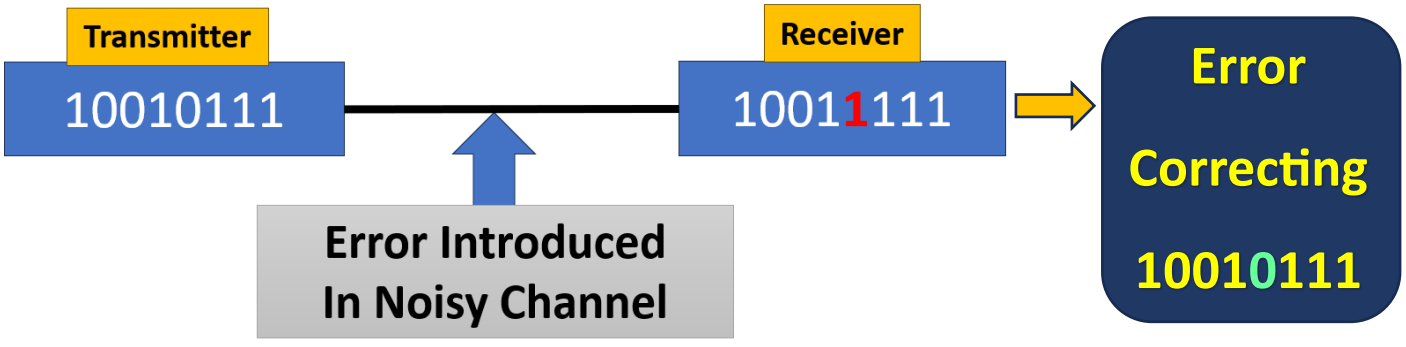


Fig: Simple block diagram of error correcting in digital data transmission.

Relevant Theories:

Redundancy for Error Correction:

A codeword is a unit of bits that can be decoded independently. The number of bits in codeword is known as the code length. If *k* data digits are transmitted by a codeword of *n* digits, the number of check digits is *m=(n-k).* The code rate is *R = k/n*. Such a code is known as an *(n, k)* code.

If the binary code length is *n*, then a total of 2^n code words (or vertices of an *n*-dimensional hypercube) is available to assign to 2^k data words. Suppose we wish to find a code that will correct up to *t* wrong digits. In this case, if we transmit a data word $\boldsymbol{d_i}$ by using one of the codewords (or vertices) $\boldsymbol{c_j}$, then because of channel errors the received word will not be $\boldsymbol{c_j}$ but will be $\boldsymbol{c_j'}$. If the channel noise causes errors in *t* or fewer digits, then $\boldsymbol{c_j'}$ will lie somewhere inside the Hamming sphere of radius *t* centered at $\boldsymbol{c_j}$. If the code is to correct up to *t* errors, then the code must have the property that all the Hamming spheres of radius *t* centered at the codewords are nonoverlapping. This means that we must not use vertices as codewords that are within a Hamming distance *t* from any codeword. If a received word lies within a Hamming sphere of radius t centered at $\boldsymbol{c_j}$, then we decide that the true transmitted codeword was $\boldsymbol{c_j}$. This scheme is capable of correcting up to *t* errors, and d_{min}, the minimum distance between *t* error correcting codewords without overlapping, is $d_{min}=2t+1$.

Next, to find a relationship between *n* and *k*, we observe that 2^n vertices, or words, are available for 2^k data words. Thus, there are 2^n-2^k redundant vertices. The number of sequences (of n digits) that differ from a given sequence by *j* digits is the number of possible combinations of *n* things taken *j* at a time and is given by $\binom{n}{j}$.

Hence, the number of ways in which up to t errors can occur is given by, $\sum_{j=1}^{t}\binom{n}{j}$

Thus, for each codeword, we must leave $\sum_{j=1}^{t}\binom{n}{j}$ vertices unused.

Because we have 2^k codewords, we must leave a total of $2^k\sum_{j=1}^{t}\binom{n}{j}$ words unused.

Therefore, the total number of words must at least be, $2^k + 2^k\sum_{j=1}^{t}\binom{n}{j} = 2^k\sum_{j=0}^{t}\binom{n}{j}$

But the total number of words, or vertices, available is 2^n. Thus, we require,

$$2^n \geq 2^k\sum_{j=0}^{t}\binom{n}{j} \; or \; , 2^{n-k} \geq \sum_{j=0}^{t}\binom{n}{j}$$

Here, (n-k=m) is the number of check digits. So, $2^m \geq \sum_{j=0}^{t}\binom{n}{j}$

This is known as the Hamming bound. It should also be remembered that the Hamming bound is a necessary but not a sufficient condition in general. However, for single-error correcting codes, it is a necessary and sufficient condition. If some *m* satisfies the Hamming bound, it does not necessarily mean that a *t*-error correcting code of n digits can be constructed.

A code for which the inequalities become equalities is known as a perfect code. In such a code the Hamming spheres (about all the codewords) not only are nonoverlapping but they exhaust all the 2^n vertices, leaving no vertex outside spheres. Perfect codes exist in only a comparatively few cases.

Binary, single-error correcting, perfect codes are called Hamming codes. For a Hamming code, $t=1$ and $d_{min}=3$. So, we have,

$$2^m = \sum_{j=0}^{t} \binom{n}{j} = 1 + n$$

$$and, \boldsymbol{n = 2^m - 1}$$

Thus, Hamming codes are *(n, k)* codes with $n=2^m-1$ and $k= 2^m-1-m$. In general, we often write Hamming code as $(2^m-1, 2^m-1-m, m)$ code.

In this project, we use 4 parity bits.

So, the number of our message bit is $\boldsymbol{(2^4-4-1) = 11}$

And, the length of the total data bit is ***(11+4) = 15***

So, this is a (15,11,4) Hamming code system.

(15,11,4) Hamming Code System:

Limitation: This method is applicable only when error is occurred in a single bit. If there error occurs in multiple bits then this error correcting method will not be applicable there.

Working Method:

Let, the 4 parity bits are P_1, P_2, P_4, P_8

P_1 checks all the bit positions where the LSB is 1 in binary representation.

So, P_1 checks those bits position whose binary representation is in the form of (X X X 1). And, P_1 stays at the first (0001) bit position.

Similarly,

Parity Bit	Checked Bit's Binary Representation Form	Staying Position
P_2	X X 1 X	2 (0010)
P_4	X 1 X X	4 (0100)
P_8	1 X X X	8 (1000)

Suppose, our 11 message bits are:

D_{11}, D_{10}, D_9, D_8, D_7, D_6, D_5, D_4, D_3, D_2, D_1

So, total transferred bits will be:

D_{11}	D_{10}	D_9	D_8	D_7	D_6	D_5	P_8	D_4	D_3	D_2	P_4	D_1	P_2	P1
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1

If we place those data bits in 4x4 table, then

	0001^{1} P_1	0010^{2} P_2	0011^{3} D_1
0100^{4} P_4	0101^{5} D_2	0110^{6} D_3	0111^{7} D_4
1000^{8} P_8	1001^{9} D_5	1010^{10} D_6	1011^{11} D_7
1100^{12} D_8	1101^{13} D_9	1110^{14} D_{10}	1111^{15} D_{11}

Every parity bit will check its corresponding message bits and take a value between 1 and 0 so that even number of 1 will be present in every region.

	0001^{1} P_1	0010^{2} P_2	0011^{3} D_1
0100^{4} P_4	0101^{5} D_2	0110^{6} D_3	0111^{7} D_4
1000^{8} P_8	1001^{9} D_5	1010^{10} D_6	1011^{11} D_7
1100^{12} D_8	1101^{13} D_9	1110^{14} D_{10}	1111^{15} D_{11}

There will be even number of '1' in this highlighted region where each data bit will be checked by P_1.

So, $P_1=D_1 \oplus D_2 \oplus D_4 \oplus D_5 \oplus D_7 \oplus D_9 \oplus D_{11}$

Similarly,

	0001^{1} P_1	0010^{2} P_2	0011^{3} D_1
0100^{4} P_4	0101^{5} D_2	0110^{6} D_3	0111^{7} D_4
1000^{8} P_8	1001^{9} D_5	1010^{10} D_6	1011^{11} D_7
1100^{12} D_8	1101^{13} D_9	1110^{14} D_{10}	1111^{15} D_{11}

$P_2=D_1 \oplus D_3 \oplus D_4 \oplus D_6 \oplus D_7 \oplus D_{10} \oplus D_{11}$

	0001^{1} P_1	0010^{2} P_2	0011^{3} D_1
0100^{4} P_4	0101^{5} D_2	0110^{6} D_3	0111^{7} D_4
1000^{8} P_8	1001^{9} D_5	1010^{10} D_6	1011^{11} D_7
1100^{12} D_8	1101^{13} D_9	1110^{14} D_{10}	1111^{15} D_{11}

$P_4=D_2 \oplus D_3 \oplus D_4 \oplus D_8 \oplus D_9 \oplus D_{10} \oplus D_{11}$

	0001^{1} P_1	0010^{2} P_2	0011^{3} D_1
0100^{4} P_4	0101^{5} D_2	0110^{6} D_3	0111^{7} D_4
1000^{8} P_8	1001^{9} D_5	1010^{10} D_6	1011^{11} D_7
1100^{12} D_8	1101^{13} D_9	1110^{14} D_{10}	1111^{15} D_{11}

$P_8=D_5 \oplus D_6 \oplus D_7 \oplus D_8 \oplus D_9 \oplus D_{10} \oplus D_{11}$

Let for an example, our 11 bit message signal is,

D_{11}	D_{10}	D_9	D_8	D_7	D_6	D_5	D_4	D_3	D_2	D_1
1	0	0	1	1	1	0	1	0	0	1

	0001^{1} P_1	0010^{2} P_2	1
0100^{4} P_4	0	0	1
1000^{8} P_8	0	1	1
1	0	0	1

	0	0010^{2} P_2	1
0100^{4} P_4	0	0	1
1000^{8} P_8	0	1	1
1	0	0	1

$P_1 = D_1 \oplus D_2 \oplus D_4 \oplus D_5 \oplus D_7 \oplus D_9 \oplus D_{11}$
$= 0$

	0	1	1
0100^{4} P_4	0	0	1
1000^{8} P_8	0	1	1
1	0	0	1

$P_2 = D_1 \oplus D_3 \oplus D_4 \oplus D_6 \oplus D_7 \oplus D_{10} \oplus D_{11}$
$= 1$

	0	1	1
1	0	0	1
$1000^{8}_{P_8}$	0	1	1
1	0	0	1

$P_4=D_2 \oplus D_3 \oplus D_4 \oplus D_8 \oplus D_9 \oplus D_{10} \oplus D_{11}$

$= 1$

	0	1	1
1	0	0	1
0	0	1	1
1	0	0	1

$P_8=D_5 \oplus D_6 \oplus D_7 \oplus D_8 \oplus D_9 \oplus D_{10} \oplus D_{11}$

$= 0$

So, total transferred bits are:

D_{11}	D_{10}	D_9	D_8	D_7	D_6	D_5	P_8	D_4	D_3	D_2	P_4	D_1	P_2	P1
1	0	0	1	1	1	0	0	1	0	0	1	1	1	0

	0	1	1
1	0	0	1
0	0	1	1
1	0	0	1

Suppose, for noisy channel, error has occurred at position 5.

D_{11}	D_{10}	D_9	D_8	D_7	D_6	D_5	P_8	D_4	D_3	D_2	P_4	D_1	P_2	P1
1	0	0	1	1	1	0	0	1	0	1	1	1	1	0



Error!!

	0	1	1
1	1	0	1
0	0	1	1
1	0	0	1

Now, the received data will be:

D_{11}	D_{10}	D_9	D_8	D_7	D_6	D_5	P_8	D_4	D_3	D_2	P_4	D_1	P_2	P1
1	0	0	1	1	1	0	0	1	0	1	1	1	1	0

To find whether there is any error or not, we have to define 4 new variables. Let them be xP_1, xP_2, xP_4, xP_8

xP_1 will check the region of P_1 along with P_1.

xP_2, xP_4, xP_8 will also work in the same manner.

Therefore,

$xP_1 = P_1 \oplus D_1 \oplus D_2 \oplus D_4 \oplus D_5 \oplus D_7 \oplus D_9 \oplus D_{11} = 1$

	0	1	1
1	1	0	1
0	0	1	1
1	0	0	1

If the value of xP1 is '1', then there will be odd number of '1' present in the highlighted region. But, the number of '1' should be even.

Here, the value of xP_1 represents that an error has occurred in that region and the binary representation of the position where error has occurred has the form of "X X X 1"

Again,

$xP_2 = P_2 \oplus D_1 \oplus D_3 \oplus D_4 \oplus D_6 \oplus D_7 \oplus D_{10} \oplus D_{11} = 0$

Here, the value of xP_2 represents that there is no error in that region. So, the place where the error has occurred has not the binary representation of the form "X X 1 X".

Therefore, binary representation of the place where the error has occurred has '0' in the second position.

	0	1	1
1	1	0	1
0	0	1	1
1	0	0	1

Similarly,

$xP_4 = P_4 \oplus D_2 \oplus D_3 \oplus D_4 \oplus D_8 \oplus D_9 \oplus D_{10} \oplus D_{11} = 1$

$xP_8 = P_8 \oplus D_5 \oplus D_6 \oplus D_7 \oplus D_8 \oplus D_9 \oplus D_{10} \oplus D_{11} = 0$

xPi	Value	Comment on the binary representation of the place where error has occurred
xP_1	1	has '1' at the 1st bit
xP_2	0	has '0' at the 2nd bit
xP_4	1	has '1' at the 3rd bit
xP_8	0	has '0' at the 4th bit

Now, if we combine all the four comments or we place the value of all xP_1, xP_2, xP_4, xP_8 serially, we will get the binary representation of the place where error has occurred.

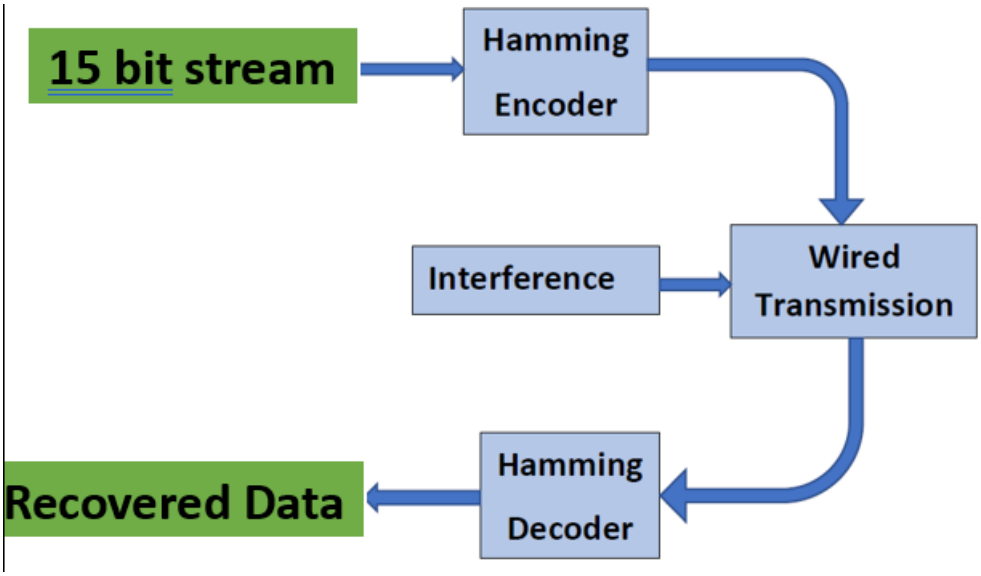
Here, that is $\mathbf{(1001)_2=(5)_{10}}$

So, error has occurred at the 5th bit position.

After that, if we toggle the 5th bit using a 4-to-16-bit decoder followed by X-OR gates, we will get the corrected and exactly the same message signal which was transmitted by the transmitter.

In the same way, we will be able to detect error in any position and correct that to get the same codeword which was transmitted.

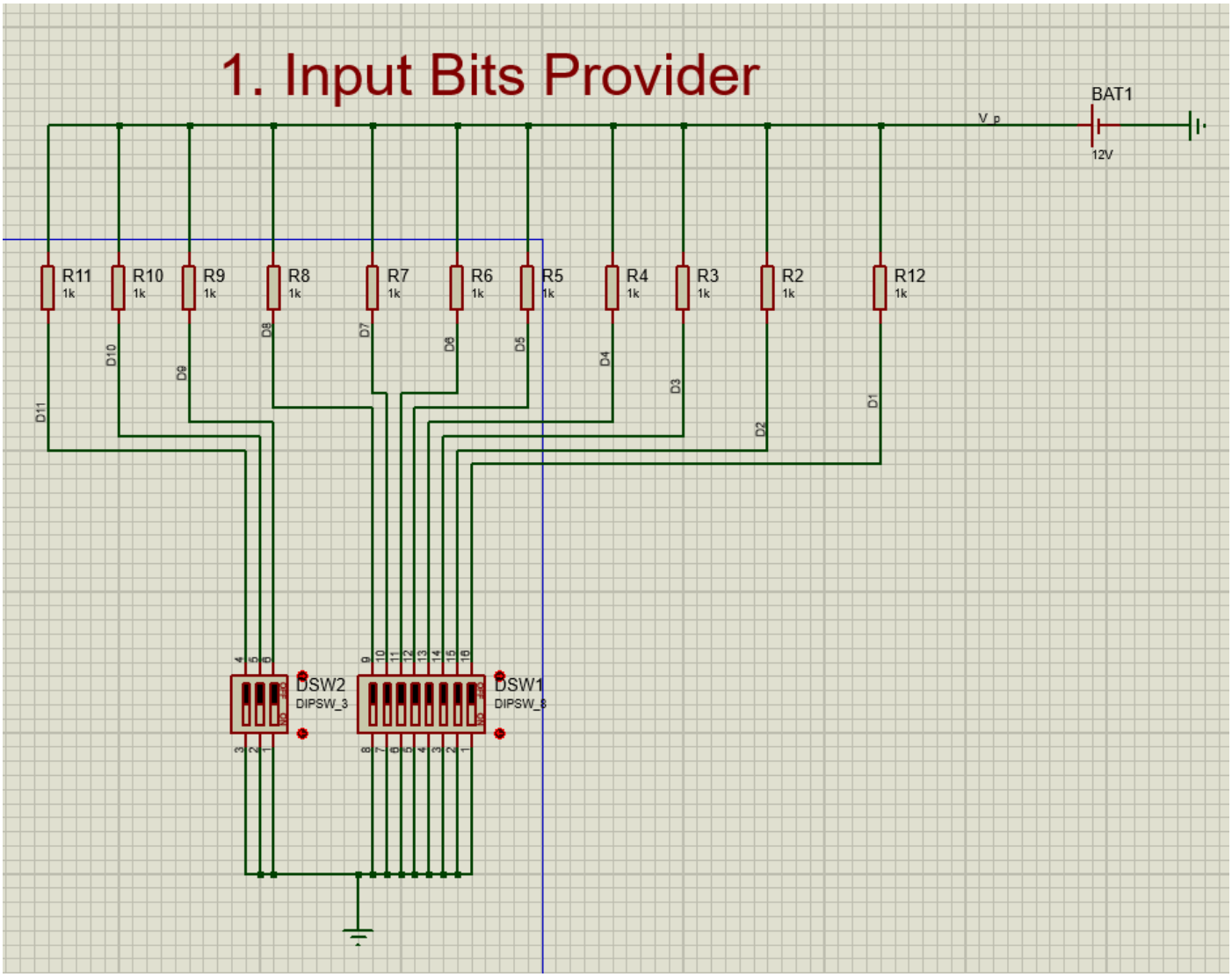
Flow Diagram:



Design:

Schematic Diagram:

Here, we provide a Proteus schematic diagram of the entire circuit.



When the switch is ‘ON’ the input for the corresponding message bit will be ‘1’ and when the switch is ‘ON’ it will provide ‘0’ as the input.

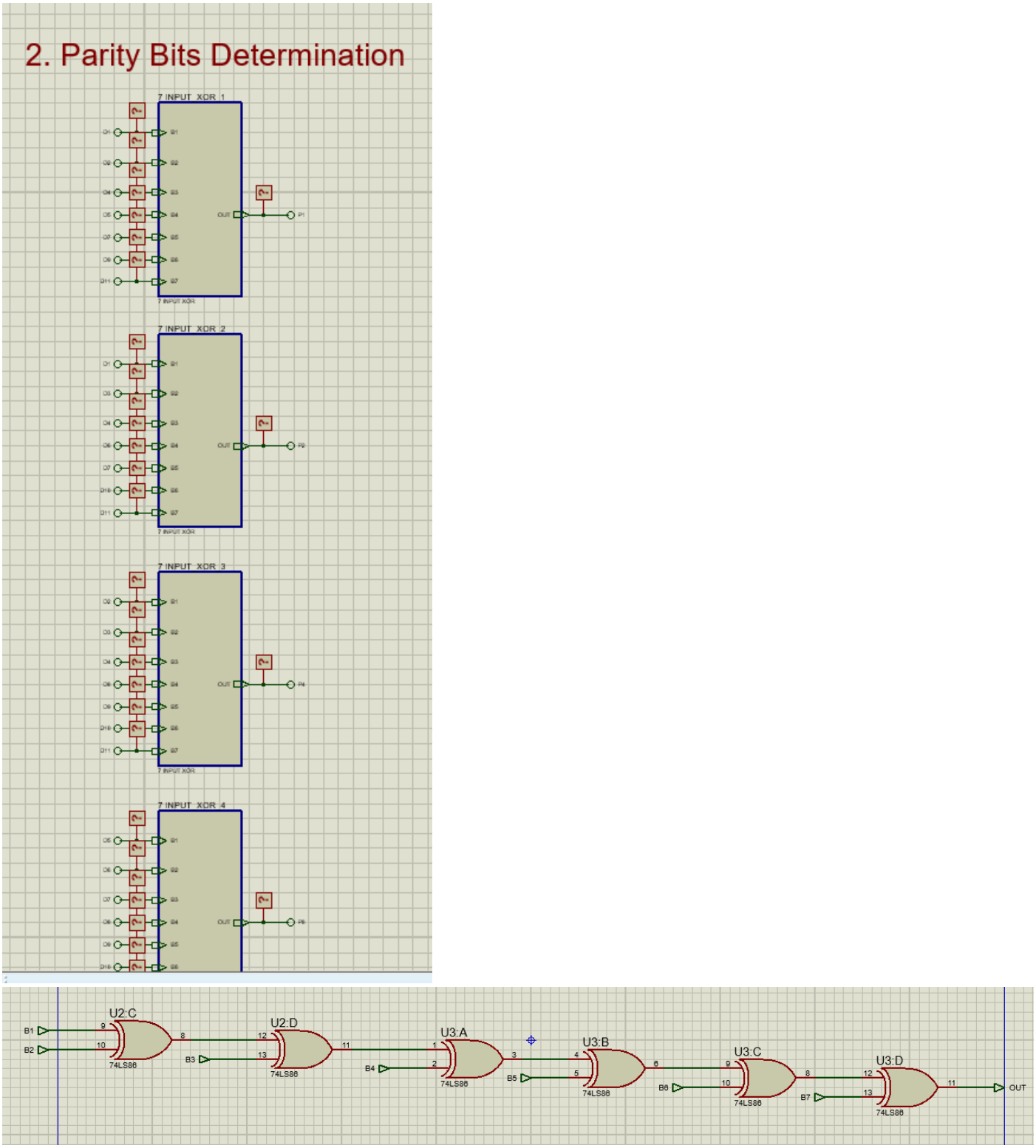
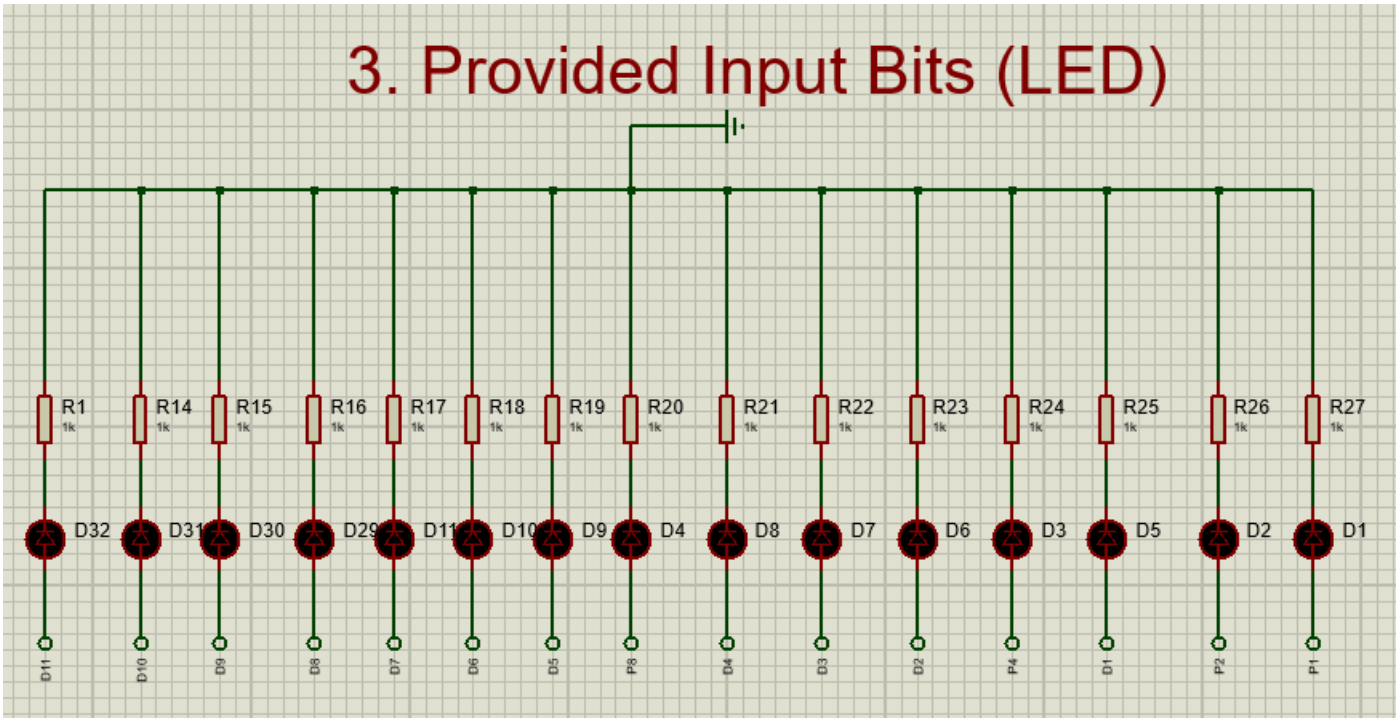
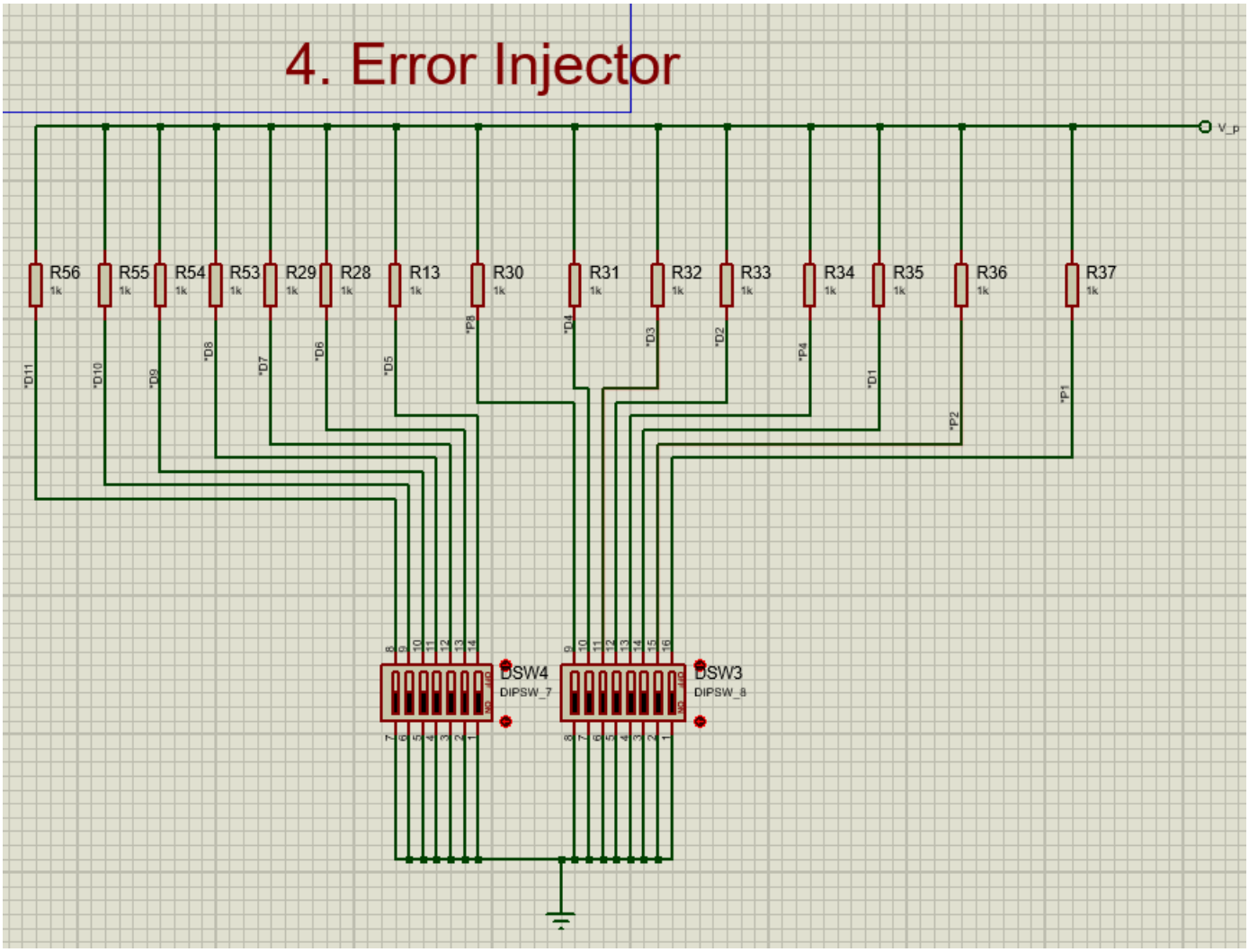
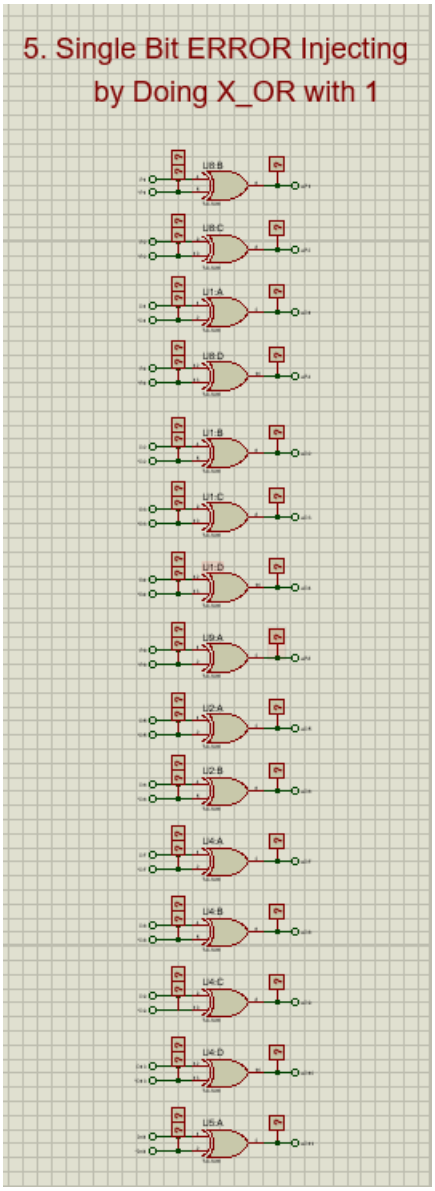


Fig: A subcircuit for determining X-OR of 7 inputs.



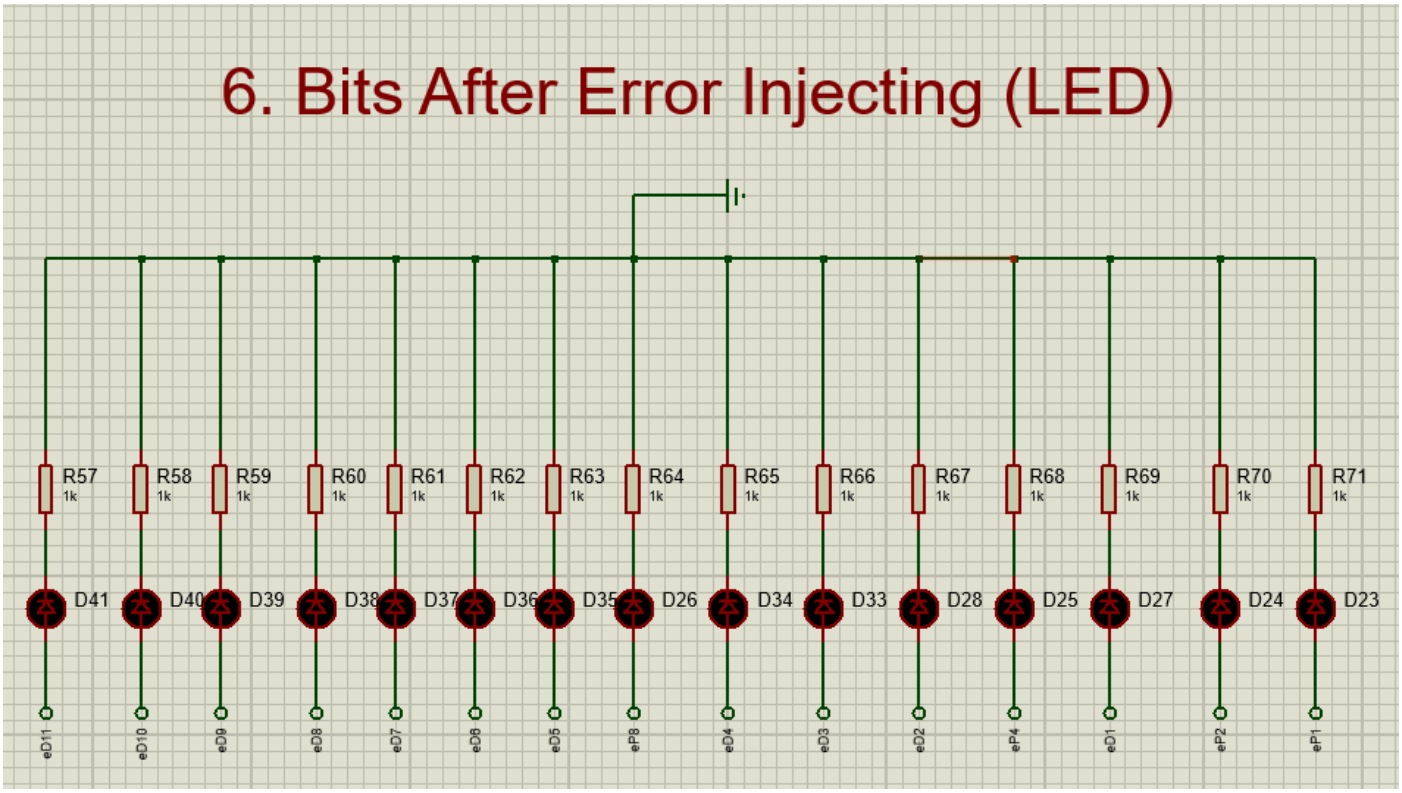
To simulate a noisy channel, we have made up a circuit block which we used to inject error in different bit positions as we wish.



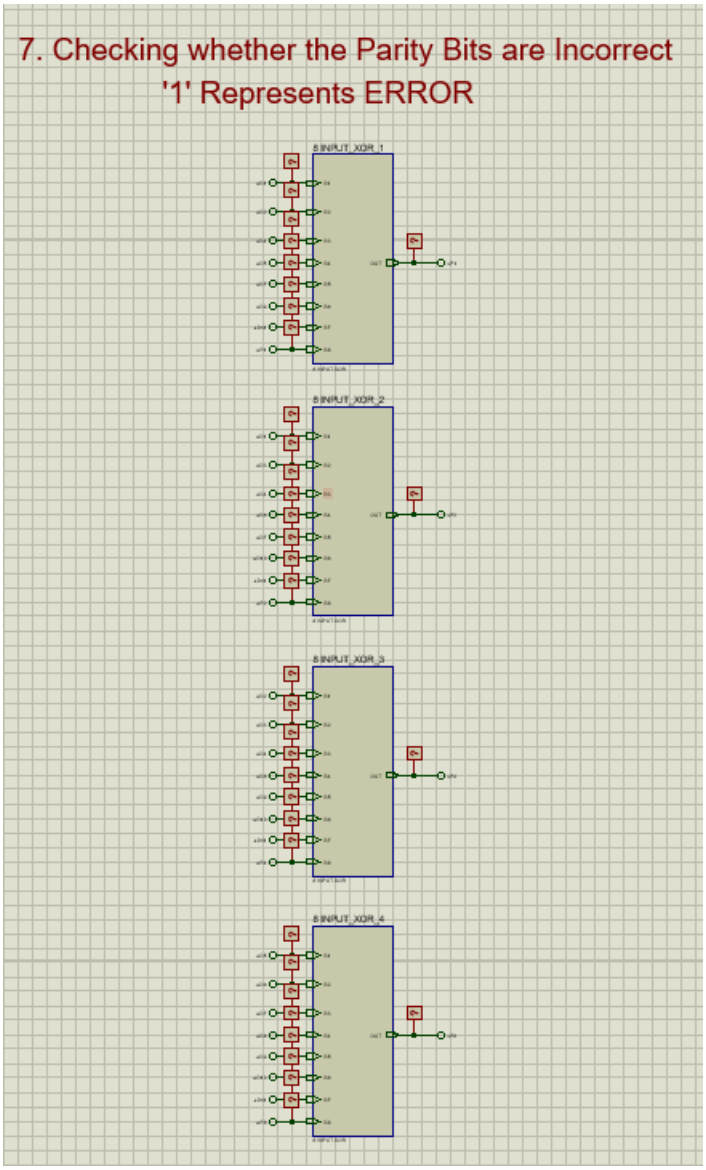


Here, X-OR operation has been done between every message bit and corresponding error injecting bit. If the error injecting bit is '1' then the input data bit will be toggled, otherwise it will be remained unchanged.

6. Bits After Error Injecting (LED)



7. Checking whether the Parity Bits are Incorrect '1' Represents ERROR



Here, we find the value of xP_1, xP_2, xP_4 and xP_8

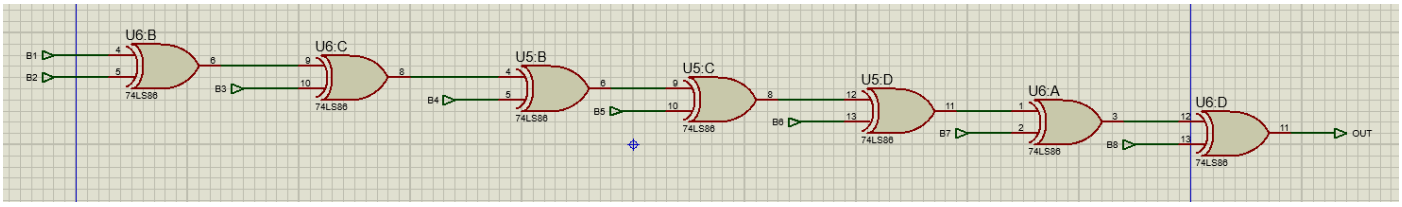
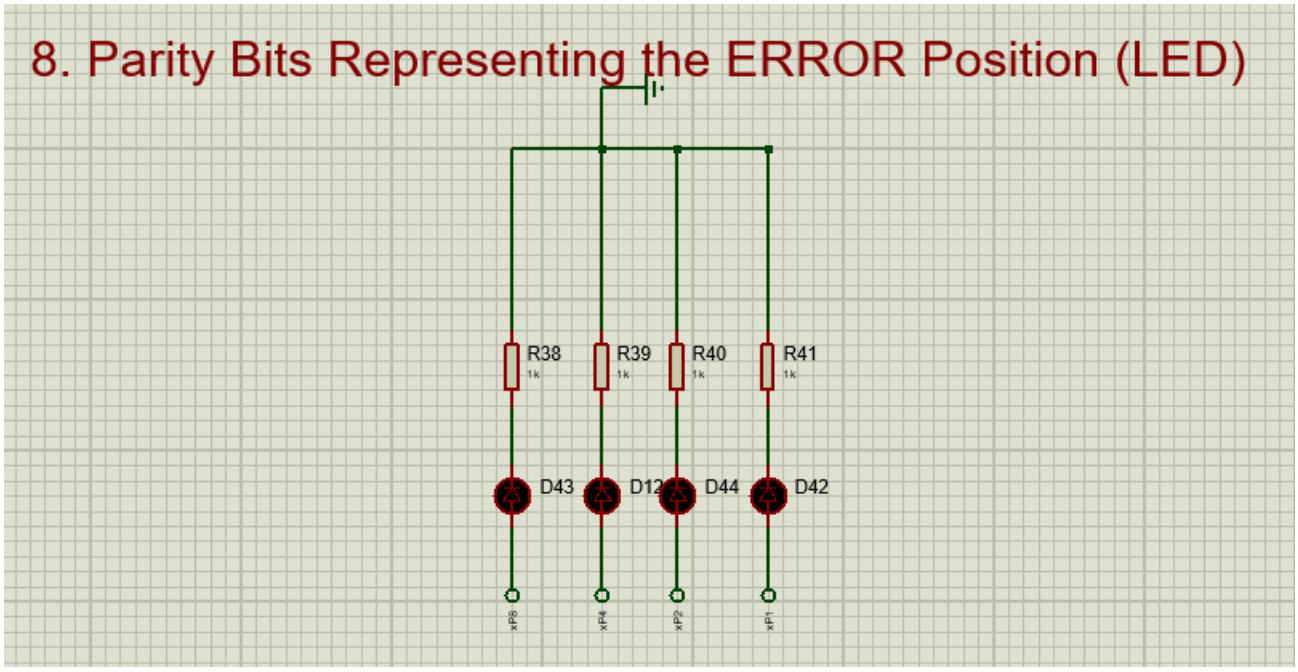
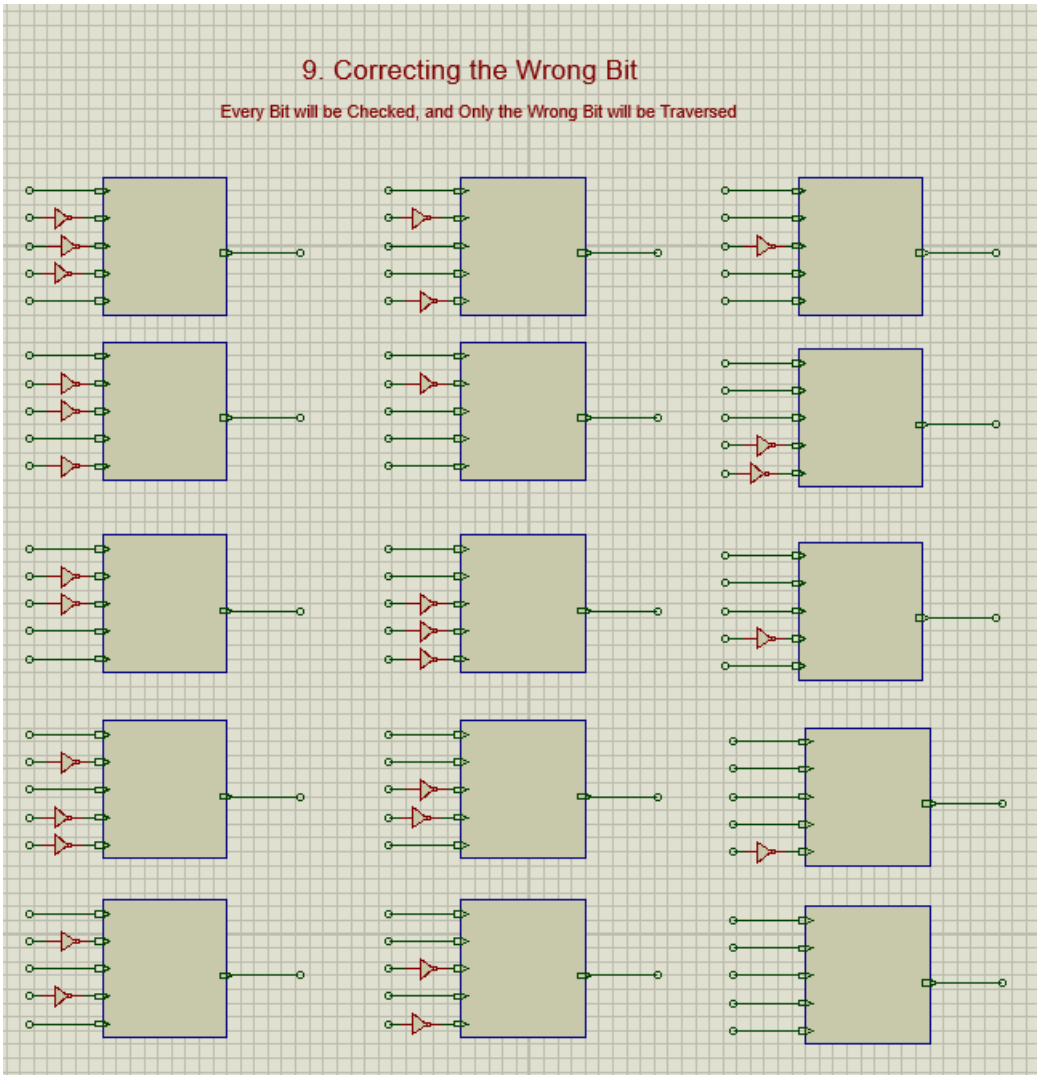
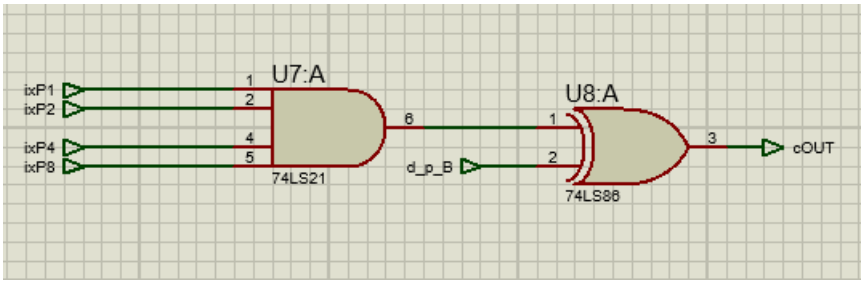


Fig: A subcircuit for determining X-OR of 8 inputs.

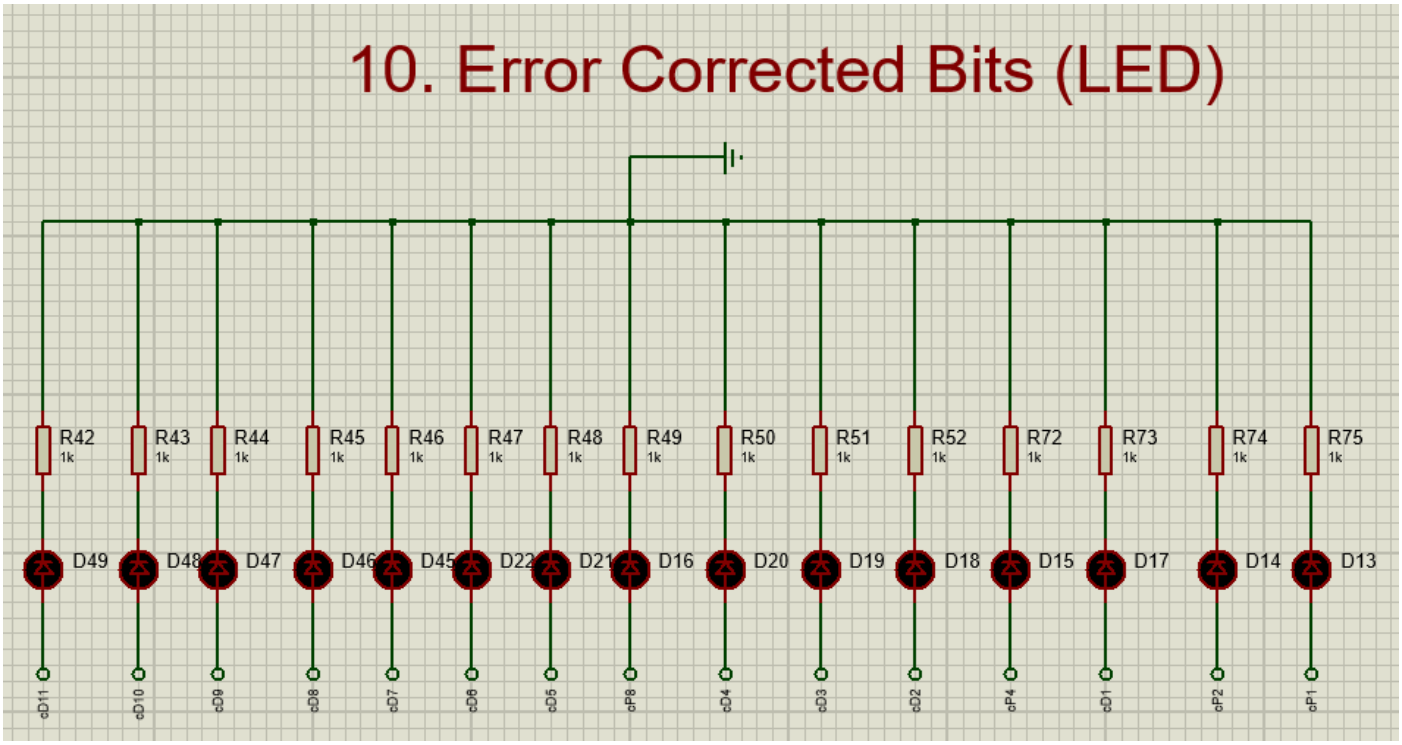


After finding the value of xP_1, xP_2, xP_4 and xP_8, we use those value in 4-to-16-bit decoder. Where the output for the place where error is occurred will be 1 and all the outputs for other positions will be 0. After that, if we send the data after error occurred and the result of 4-to-16-bit decoder, then only where the error has occurred will be toggled and other bits will remain unchanged. Thus, we will correct the erroneous bit and be able to get the message which was transmitted.





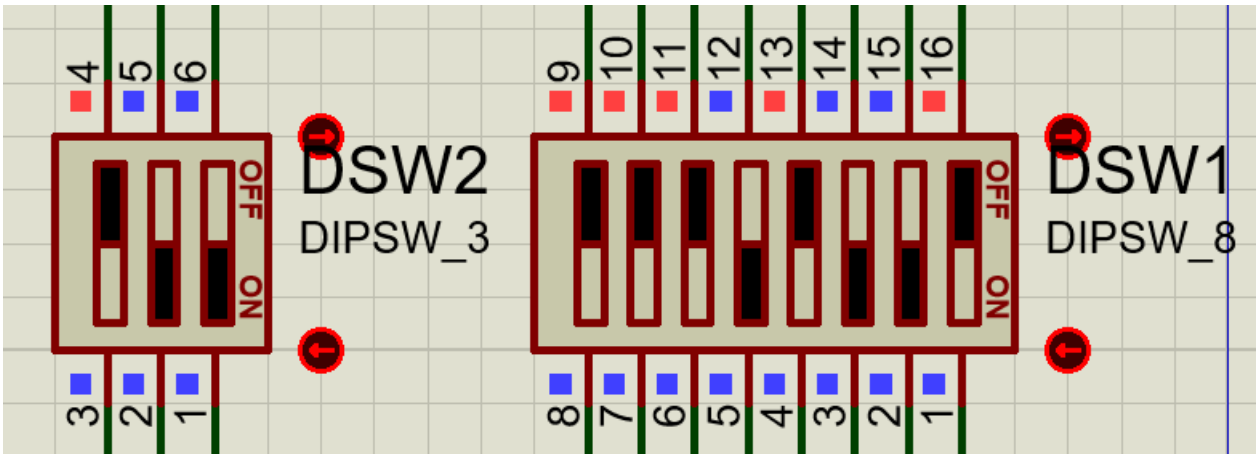
In this part, we get the output of a 4-to-16-bit decoder through a 4 input AND gate. Then, we send the send the data after error occurred and the result of 4-to-16-bit decoder to get the corrected value.



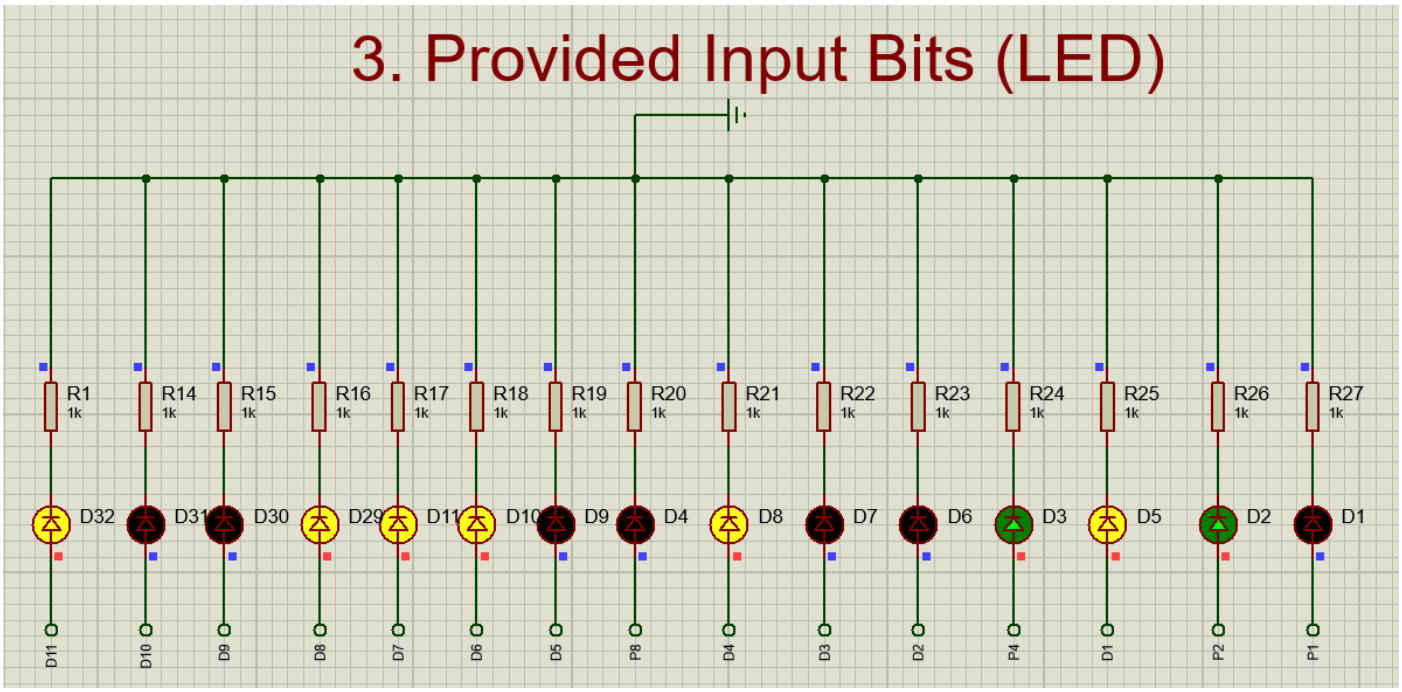
Simulation Output:

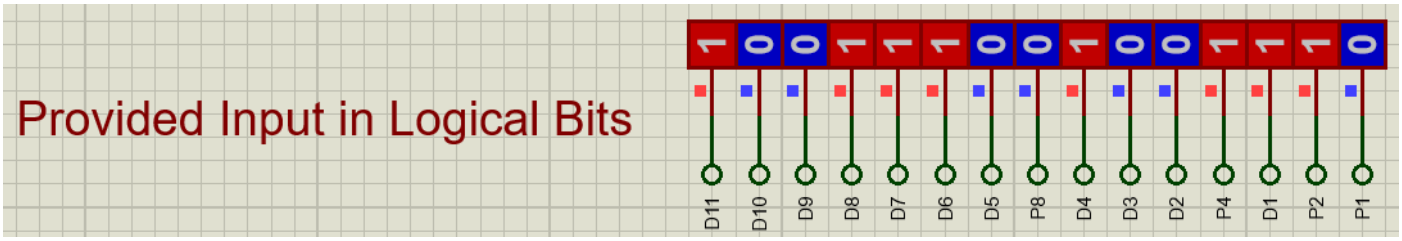
Simulation output for the example used in the working method part of (15,11,4) Hamming Code system shown below.

Input Switch:



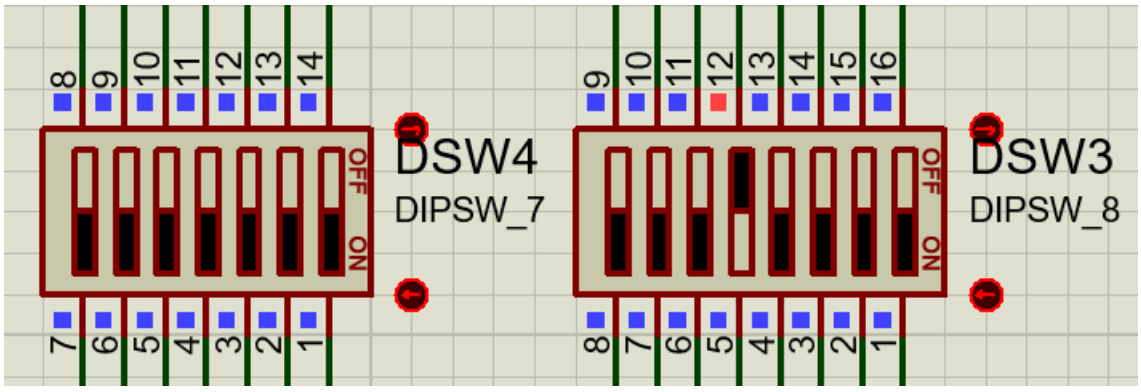
Input Bits:



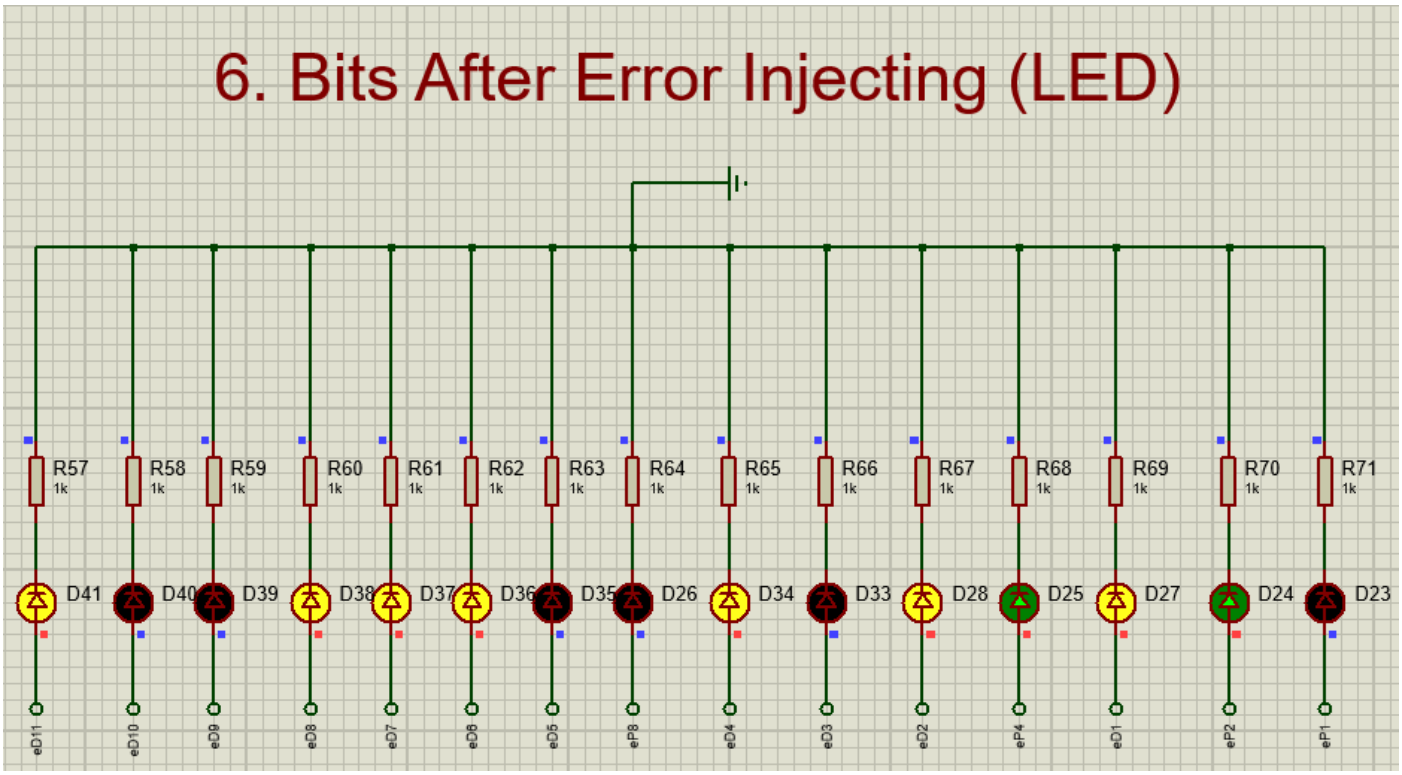


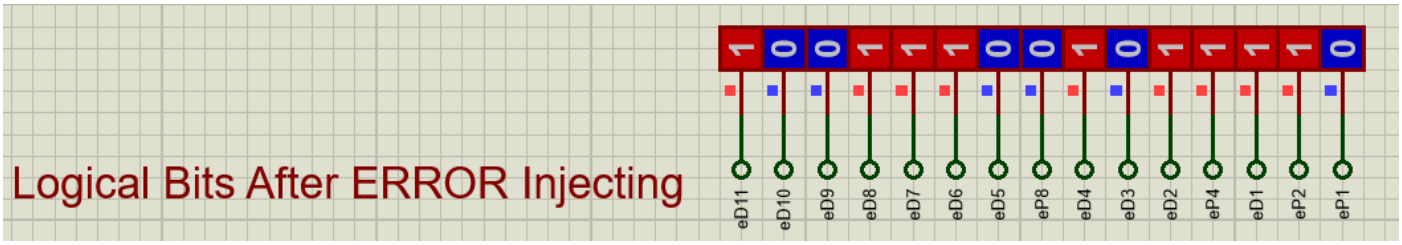
Error Injecting Switch:

Error injected at 5th bit position.

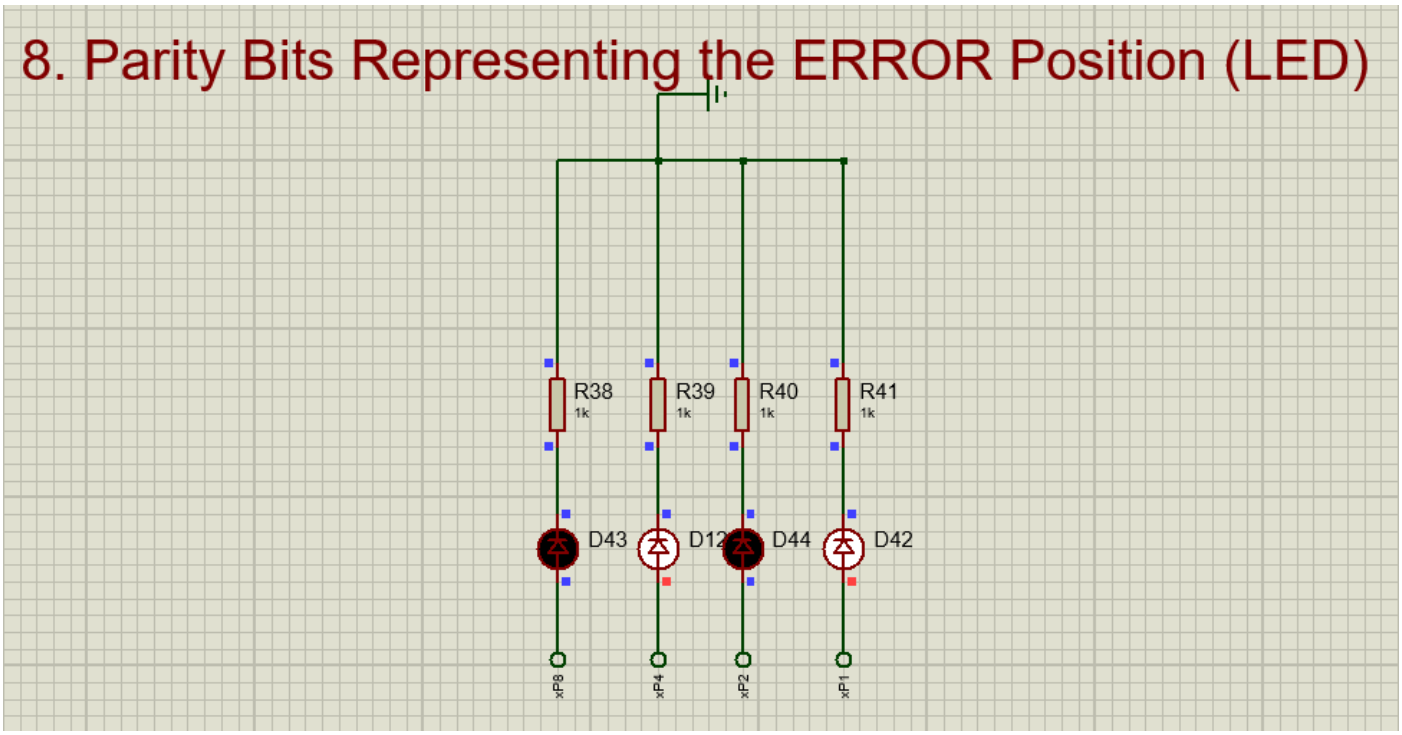


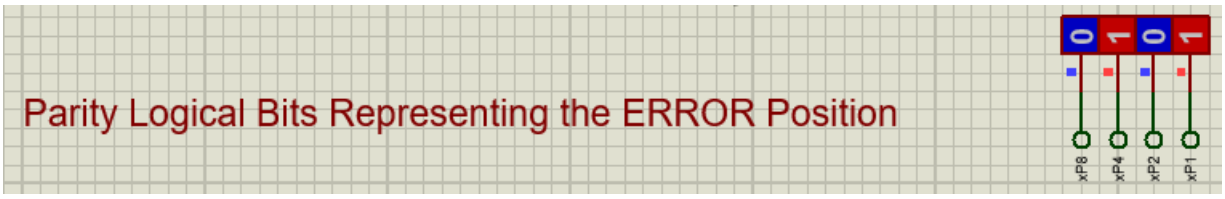
Bits After Error Injecting:



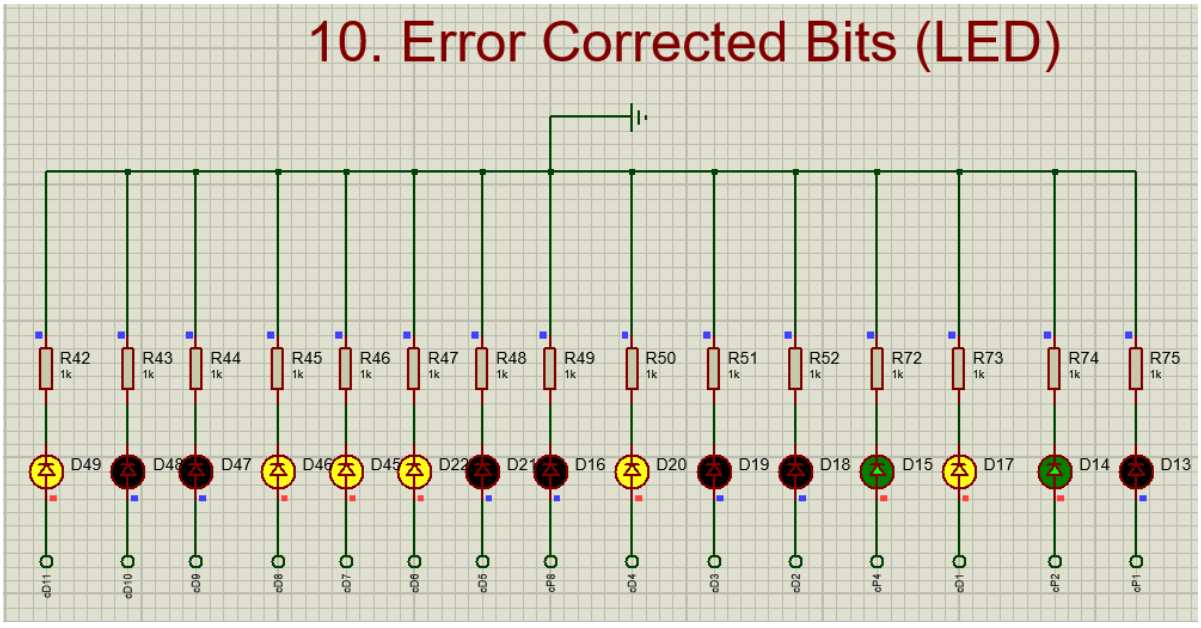


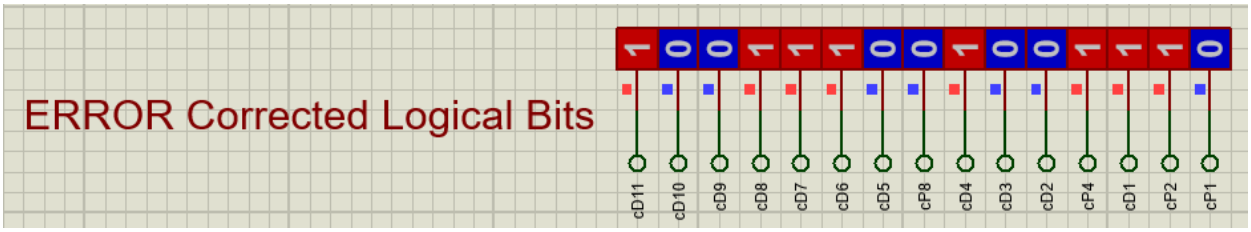
Parity Bits Representing the Error Position:





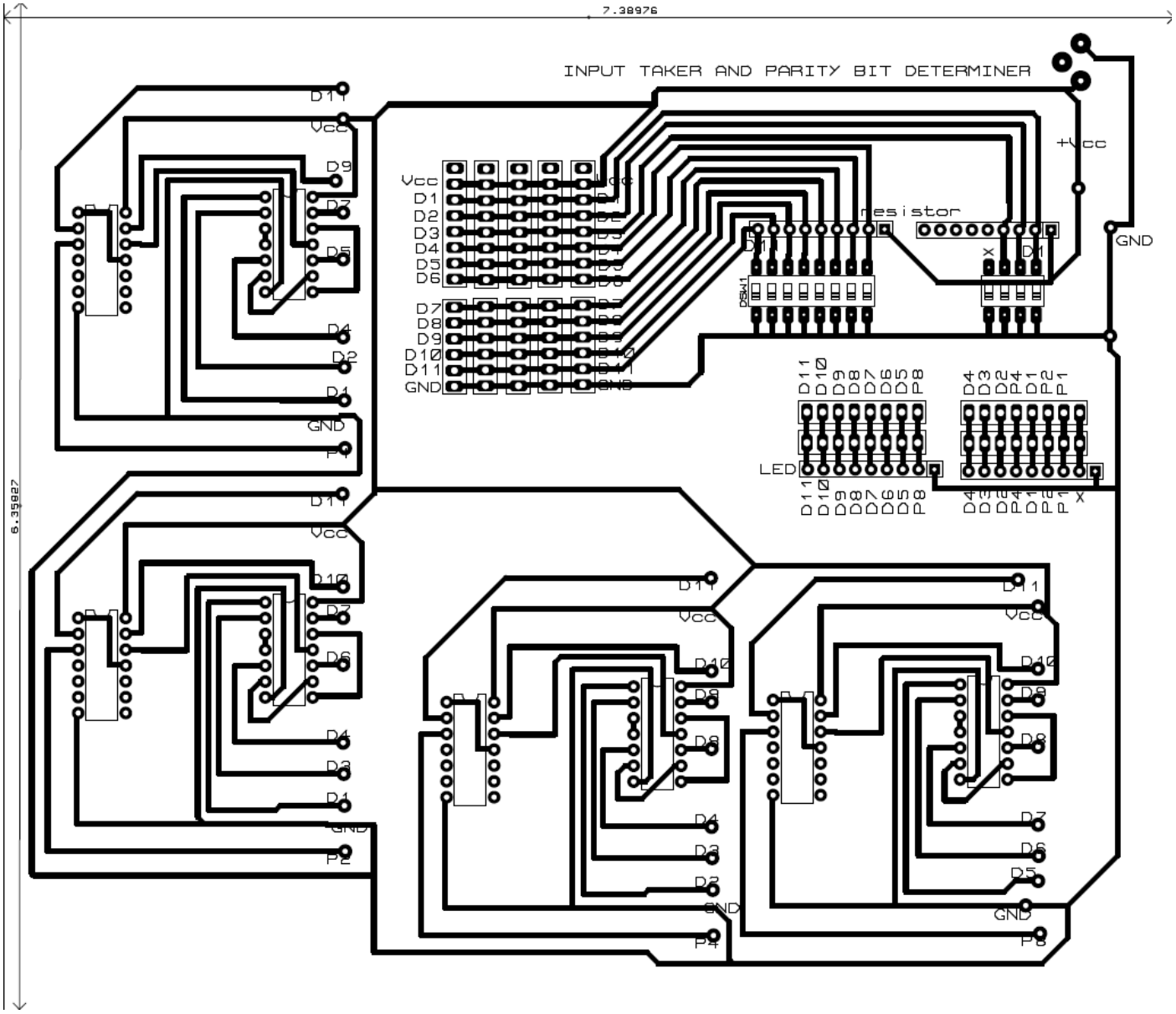
Error Corrected Bits:



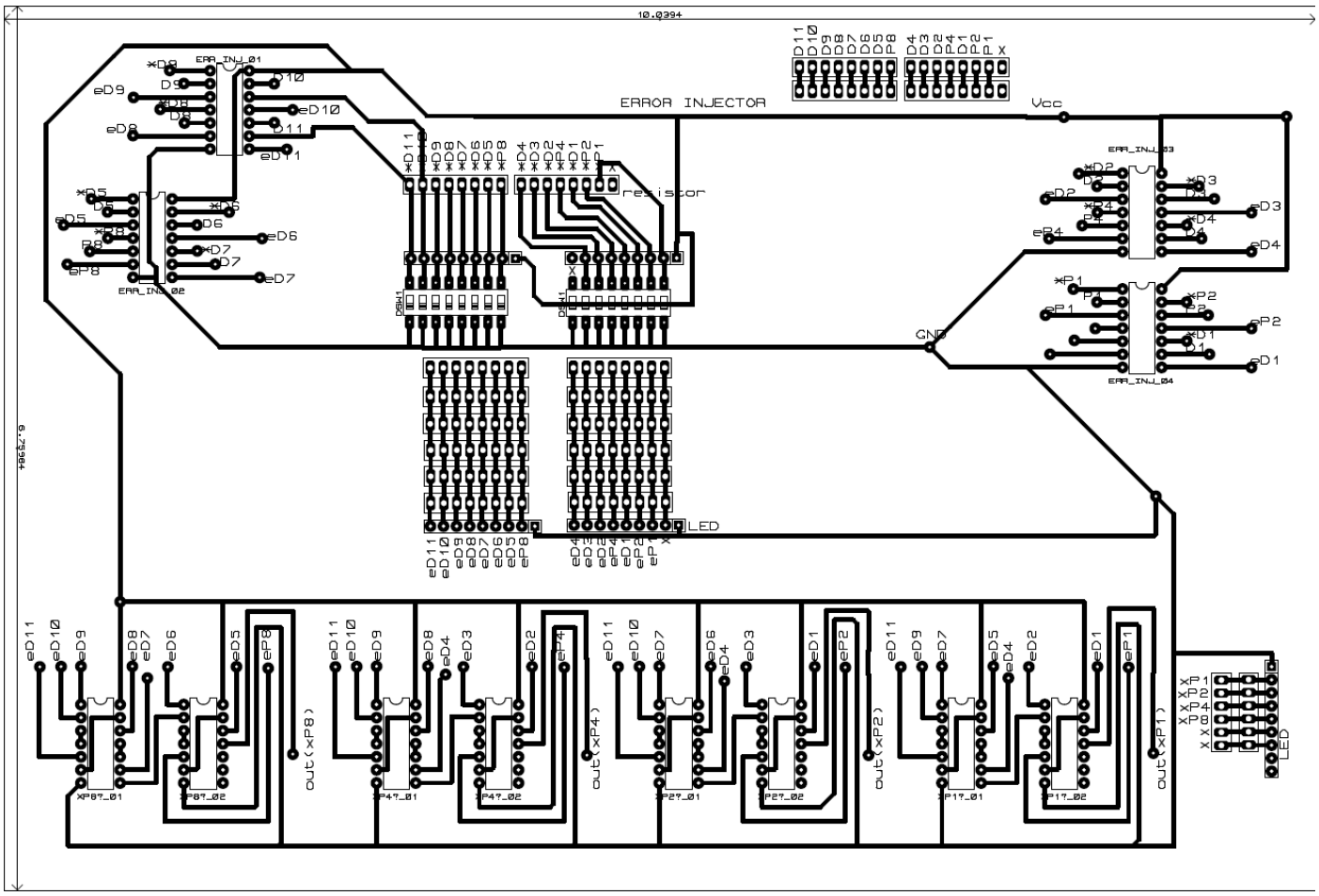


PCB Design:

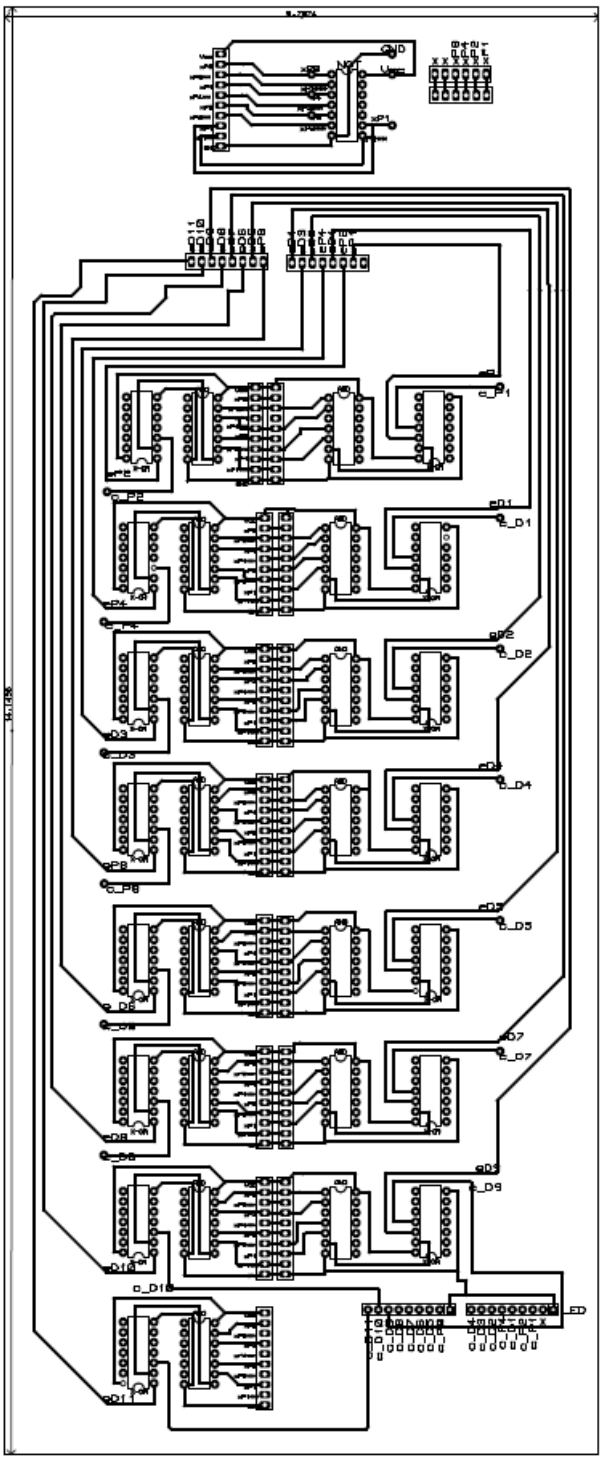
Input Taker and Parity Bit Determiner:



Error Injector:



Error Corrector:



Hardware Implementation:

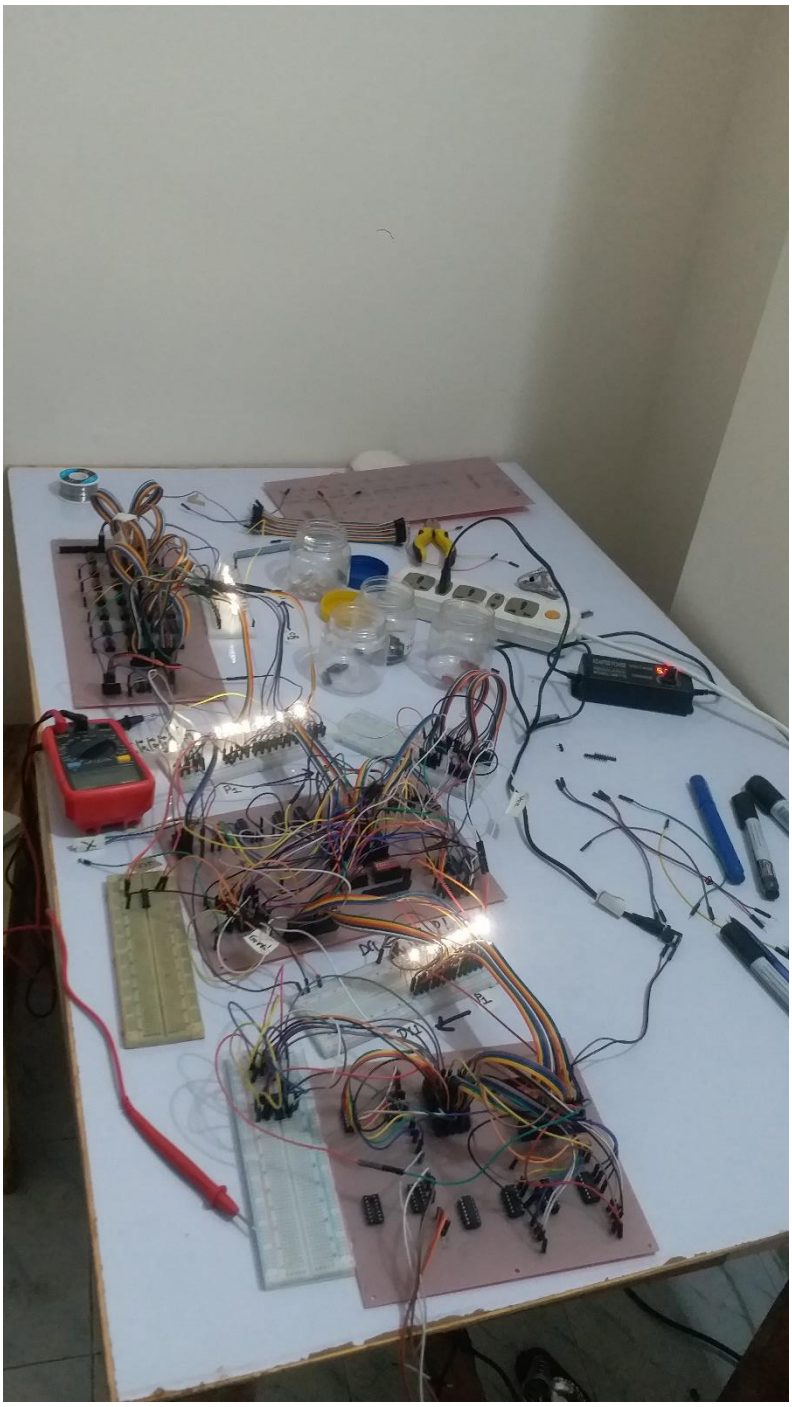


Fig: After implementing the circuit in hardware with PCB.

Testing the Circuit:

Here, outputs of three different message bits are shown.

Test 1:

Input Data:

D_{11}	D_{10}	D_9	D_8	D_7	D_6	D_5	D_4	D_3	D_2	D_1
0	**0**	**1**	**0**	**1**	**0**	**0**	**1**	**1**	**1**	**0**

	0	**1**	**0**
0	**1**	**1**	**1**
0	**0**	**0**	**1**
0	**1**	**0**	**0**

D_{11}	D_{10}	D_9	D_8	D_7	D_6	D_5	P_8	D_4	D_3	D_2	P_4	D_1	P_2	P1
0	0	1	0	1	0	0	0	1	1	1	0	0	1	0

D_{11}	D_{10}	D_9	D_8	D_7	D_6	D_5	P_8	D_4	D_3	D_2	P_4	D_1	P_2	P1
0	0	1	0	1	0	0	0	1	1	1	0	0	1	0

Error in

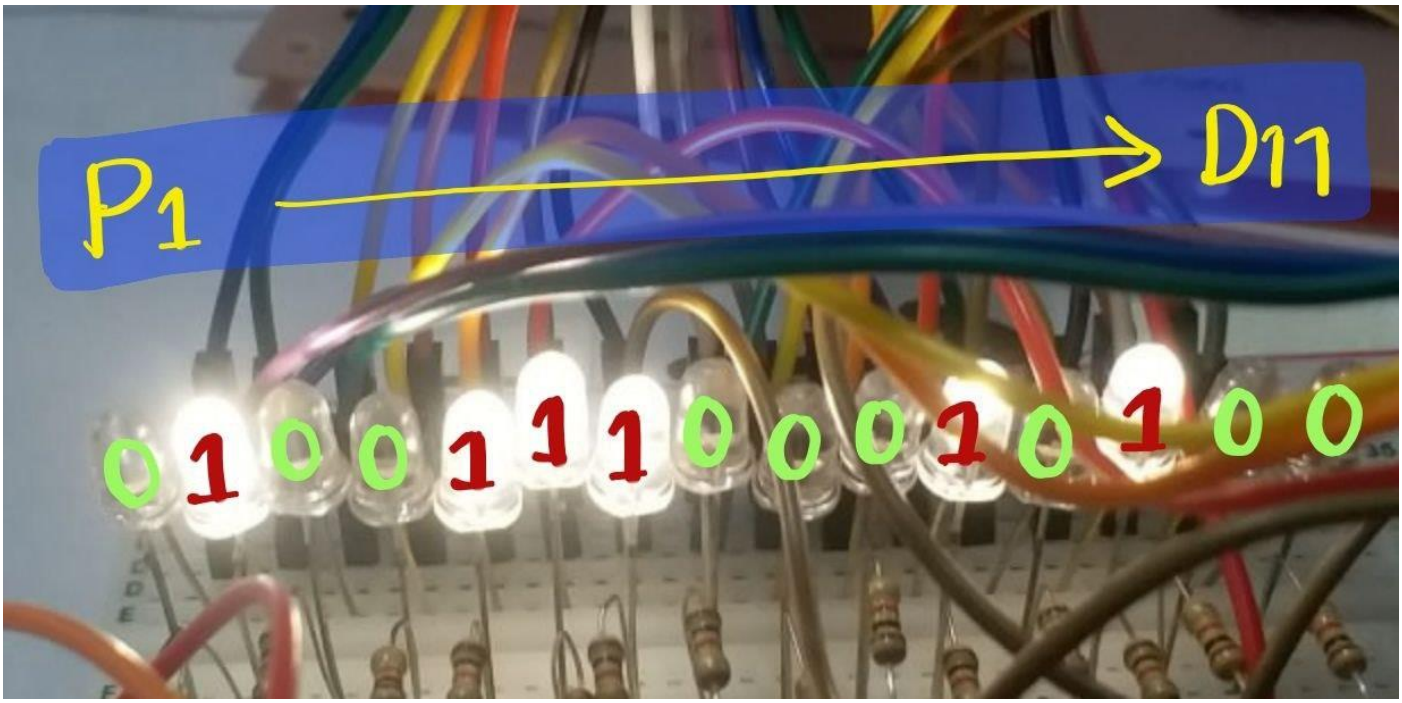
12th position

Input Switch:

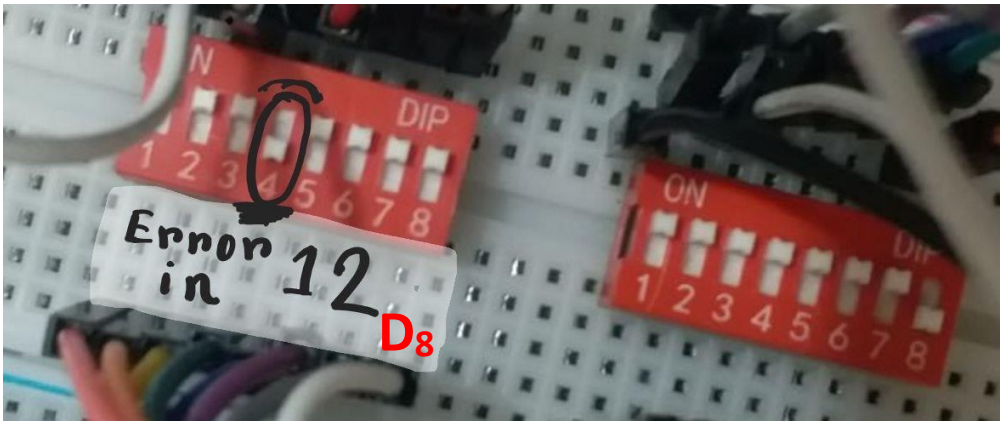
D11←←←←←D1



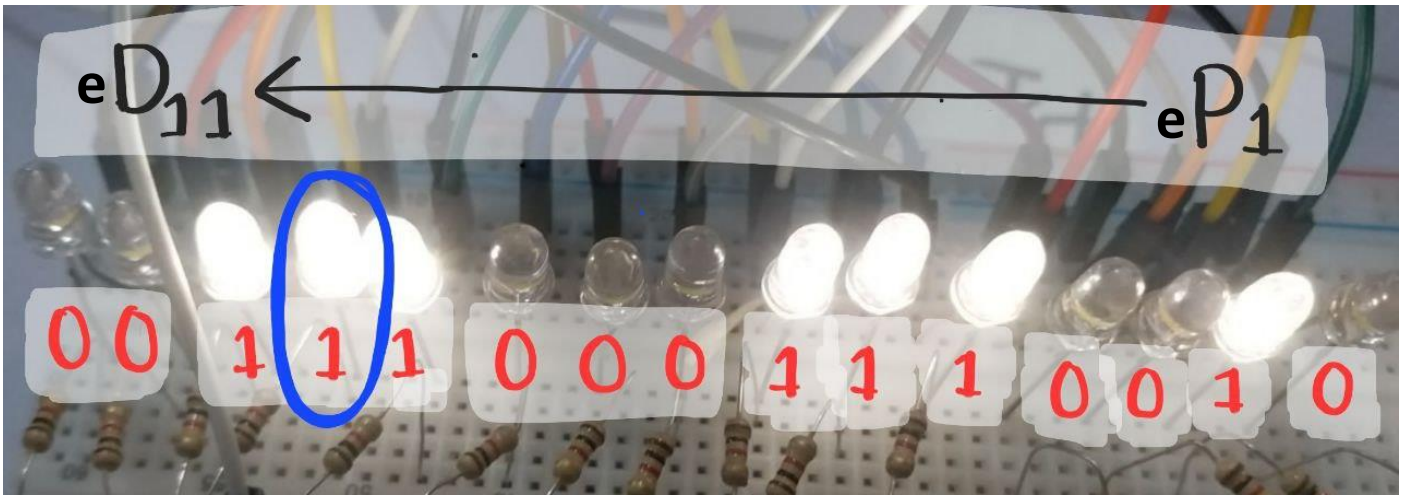
Observing the input:



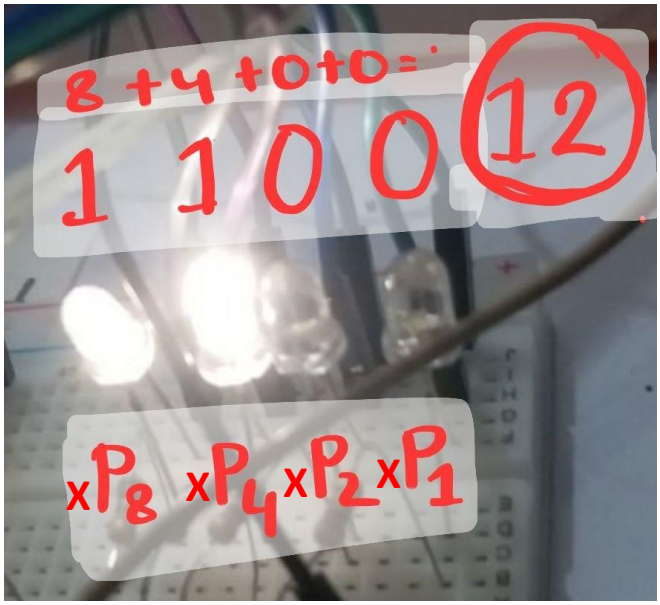
Error Injecting Switch:



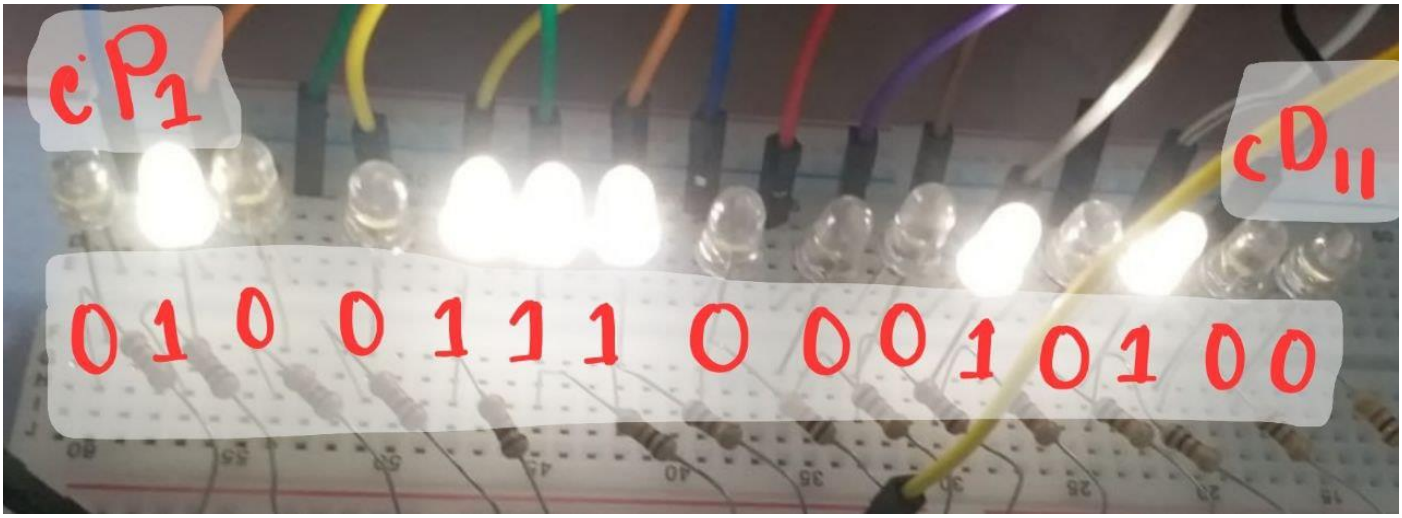
Message bits after error injecting:



Parity bits representing the error position:



Error corrected bits:



Test 2:

Input Data:

D_{11}	D_{10}	D_9	D_8	D_7	D_6	D_5	D_4	D_3	D_2	D_1
1	0	0	1	1	1	0	1	0	0	1

	0	1	1
1	0	0	1
0	0	1	1
1	0	0	1

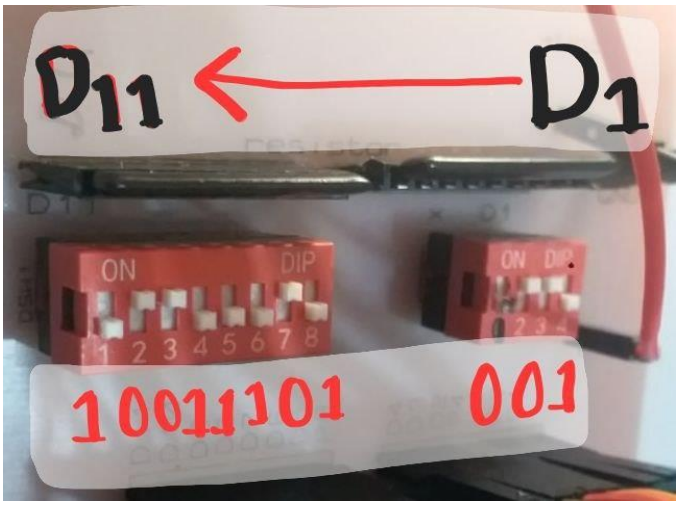
D_{11}	D_{10}	D_9	D_8	D_7	D_6	D_5	P_8	D_4	D_3	D_2	P_4	D_1	P_2	P1
1	0	0	1	1	1	0	0	1	0	0	1	1	1	0

D_{11}	D_{10}	D_9	D_8	D_7	D_6	D_5	P_8	D_4	D_3	D_2	P_4	D_1	P_2	P1
1	0	0	1	1	1	0	0	1	0	1	1	1	1	0

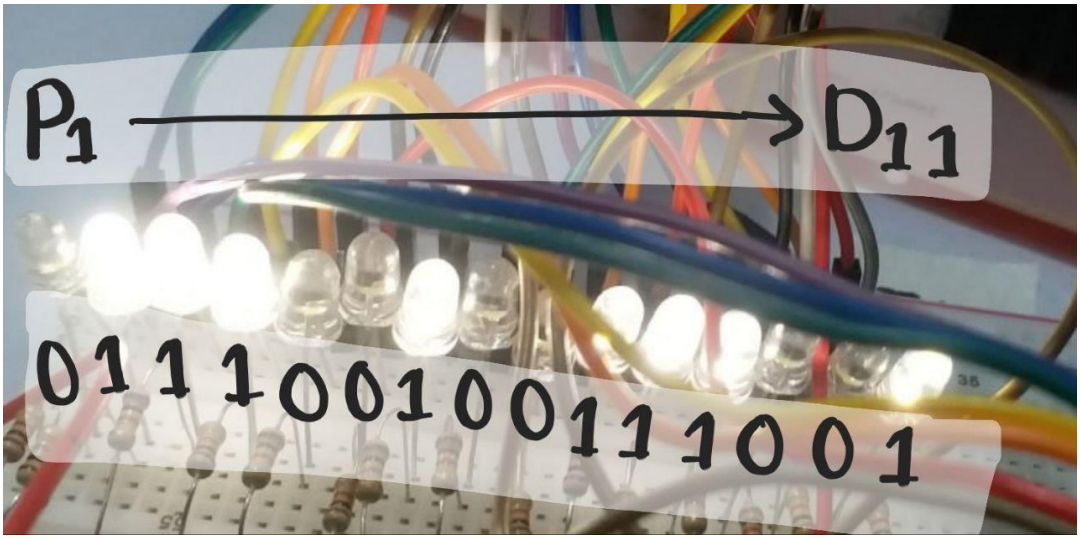
Error in

5th position

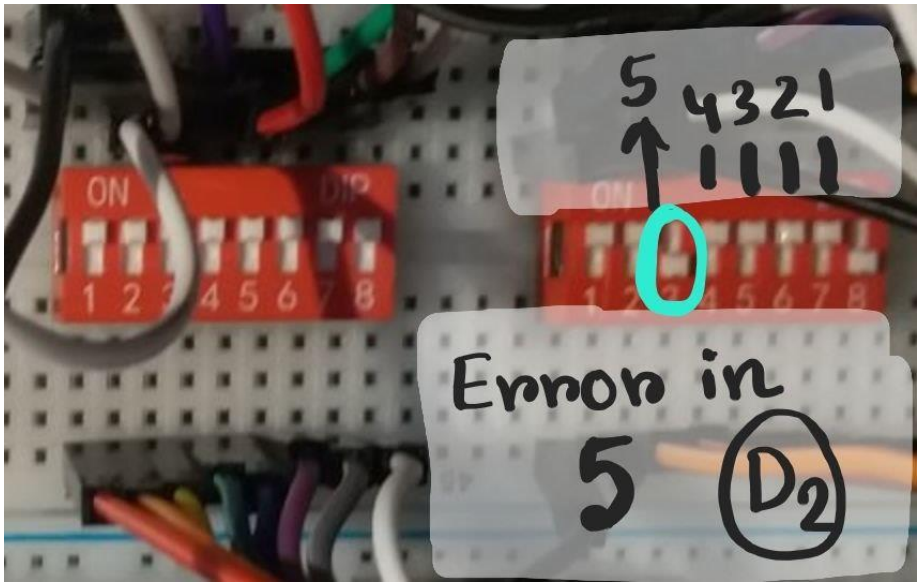
Input Switch:



Observing the input:



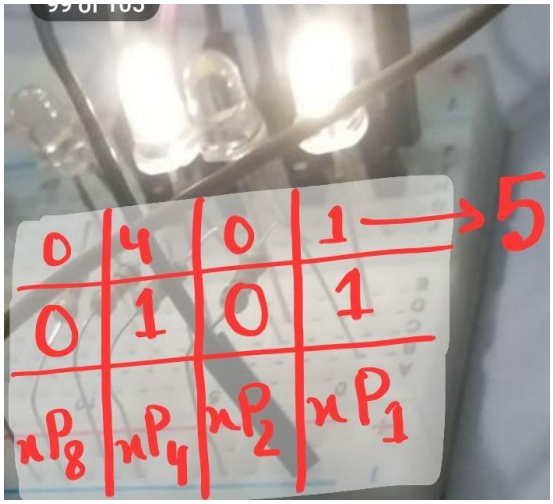
Error Injecting Switch:



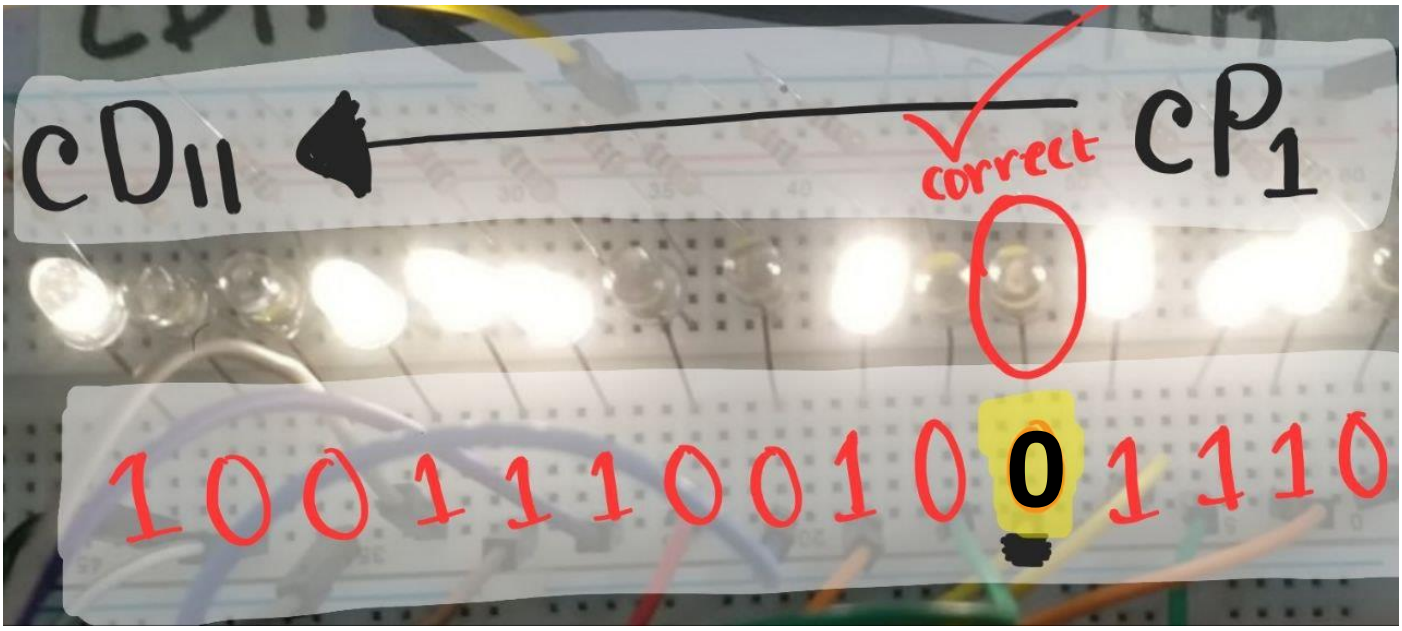
Message bits after error injecting:



Parity bits representing the error position:



Error corrected bits:



Test 3:

Input Data:

D_{11}	D_{10}	D_9	D_8	D_7	D_6	D_5	D_4	D_3	D_2	D_1
0	**0**	**1**	**0**	**0**	**1**	**1**	**1**	**0**	**1**	**1**

	1	**1**	**1**
1	**1**	**0**	**1**
1	**1**	**1**	**0**
0	**1**	**0**	**0**

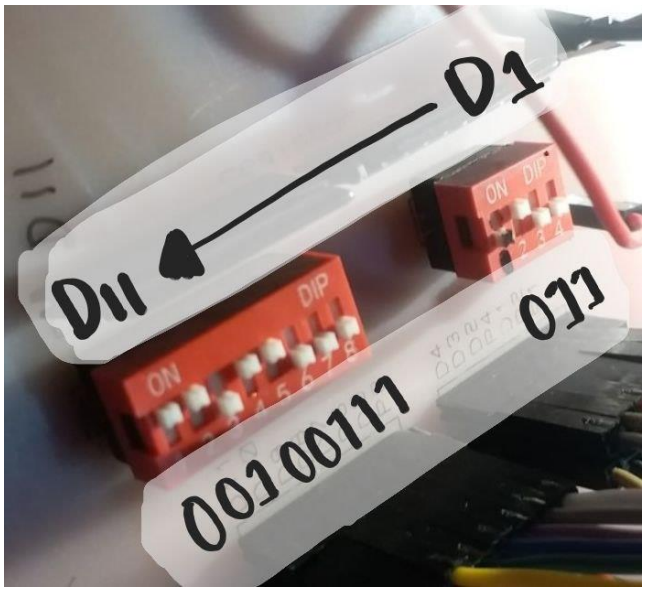
D_{11}	D_{10}	D_9	D_8	D_7	D_6	D_5	P_8	D_4	D_3	D_2	P_4	D_1	P_2	P1
0	0	1	0	0	1	1	1	1	0	1	1	1	1	1

D_{11}	D_{10}	D_9	D_8	D_7	D_6	D_5	P_8	D_4	D_3	D_2	P_4	D_1	P_2	P1
0	0	1	0	0	1	**0**	1	1	0	1	1	1	1	1

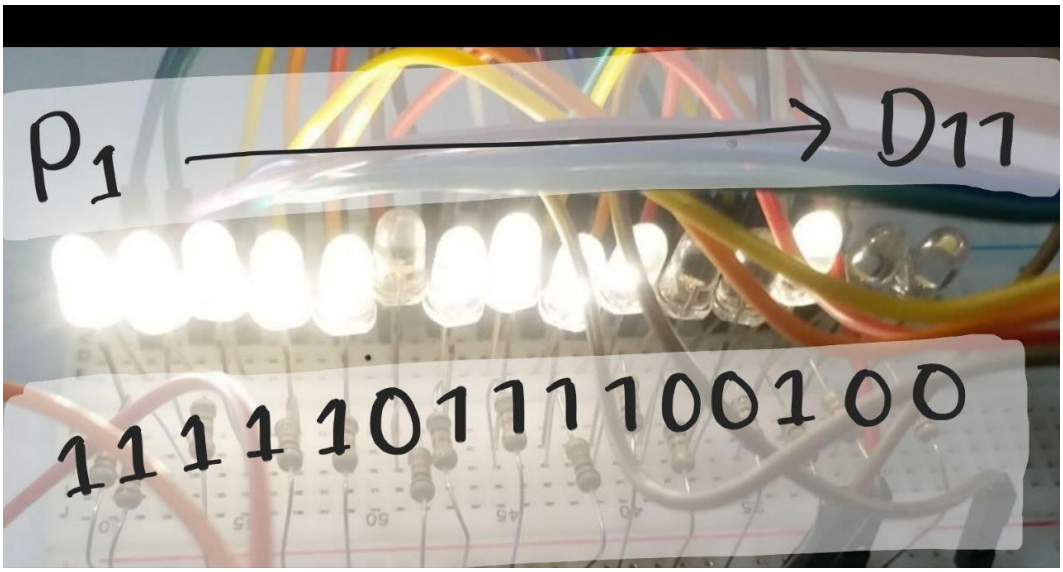
Error in

9th position

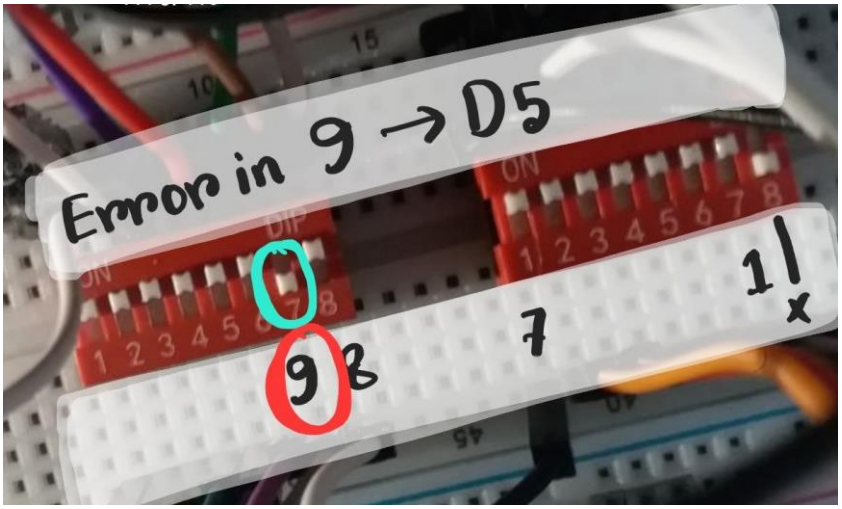
Input Switch:



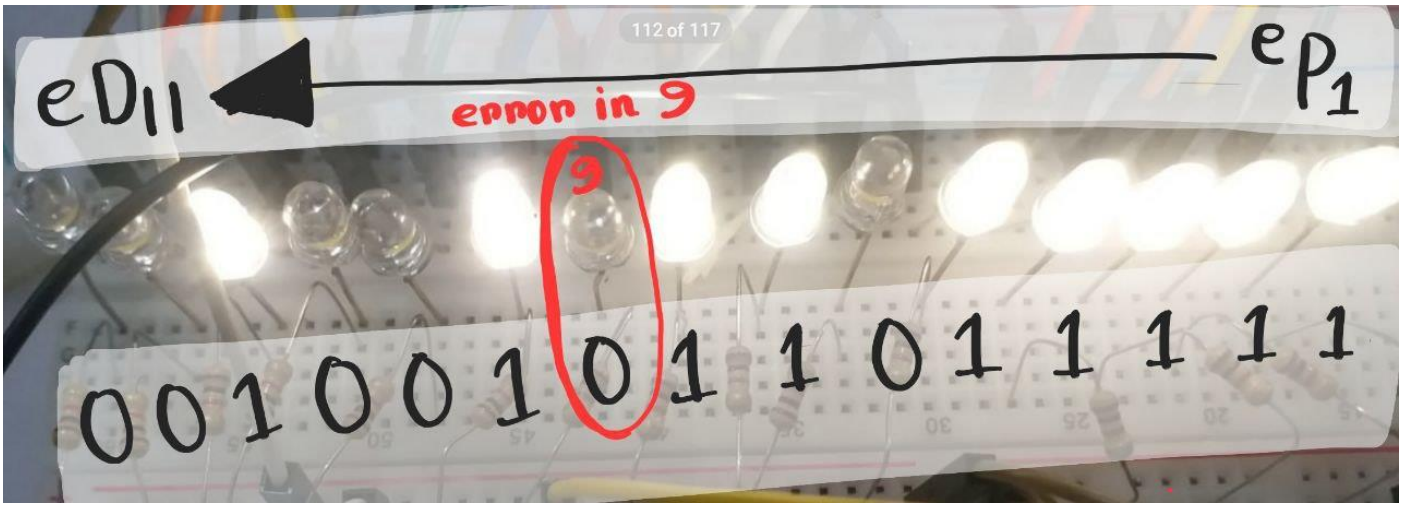
Observing the input:



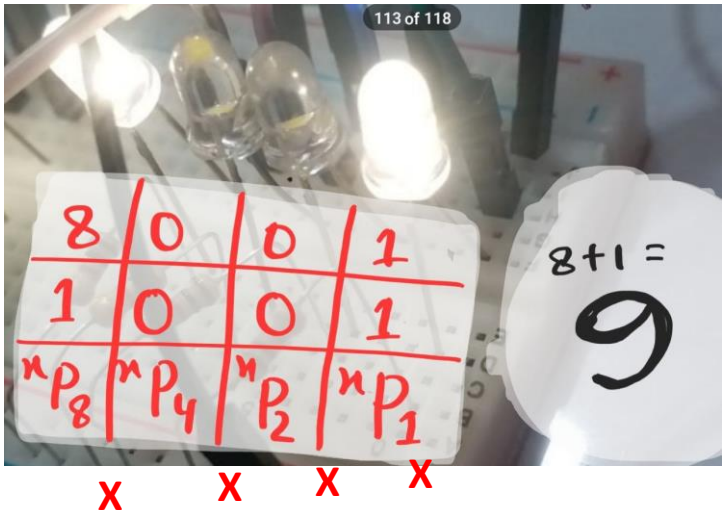
Error Injecting Switch:



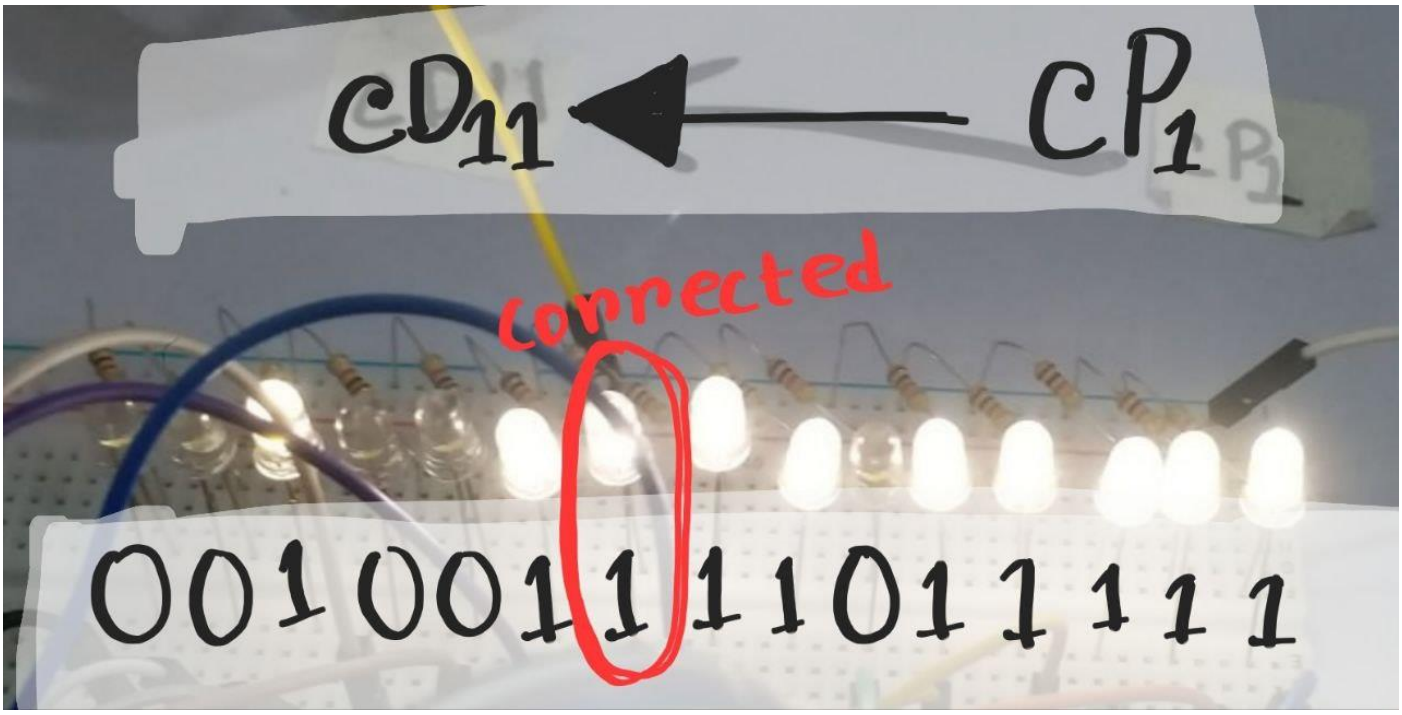
Message bits after error injecting:



Parity bits representing the error position:



Error corrected bits:



Reflection on Teamwork:

SL no.	Contribution	Student IDs
1.	Pre-planning to complete the project	2006099, 2006117, 2006118, 2006121
2.	Creating a schematic file on Proteus to simulate the entire project	2006121
3.	Schematic verifying and exploring	2006099, 2006117, 2006118, 2006121
4.	Purchasing Components for the first time	2006099, 2006121
5.	Trial on breadboard to implement the circuit in hardware (but failed)	2006099, 2006117, 2006118, 2006121
6.	PCB designing	2006121
7.	PCB ordering to TechshopBD	2006121
8.	PCB receiving after printing from TechshopBD	2006118, 2006121
9.	Purchasing Components for the second time	2006099, 2006121
10.	PCB Soldering	2006117, 2006121
11.	Implementing hardware circuits with PCB	2006099, 2006117, 2006118, 2006121
12.	Testing and photo or video capturing	2006099, 2006117, 2006118, 2006121
13.	Report Writing	2006099, 2006117, 2006118, 2006121

Individual Contribution of Each Member:

Student IDs	Contribution
2006099	Pre-planning, schematic verifying and exploring, purchasing components, trial on breadboard, implementing hardware circuits with PCB, testing and recording, report writing.
2006117	Pre-planning, schematic verifying and exploring, trial on breadboard, PCB soldering, implementing hardware circuits with PCB, testing and recording, report writing.
2006118	Pre-planning, schematic verifying and exploring, trial on breadboard, PCB receiving after printing, implementing hardware circuits with PCB, testing and recording, report writing.
2006121	Pre-planning, creating schematic, schematic verifying and exploring, purchasing components, trial on breadboard, PCB designing, PCB ordering, PCB receiving after printing, PCB Soldering, implementing hardware circuits with PCB, testing and recording, report writing.

Time-line of Project Implementation:

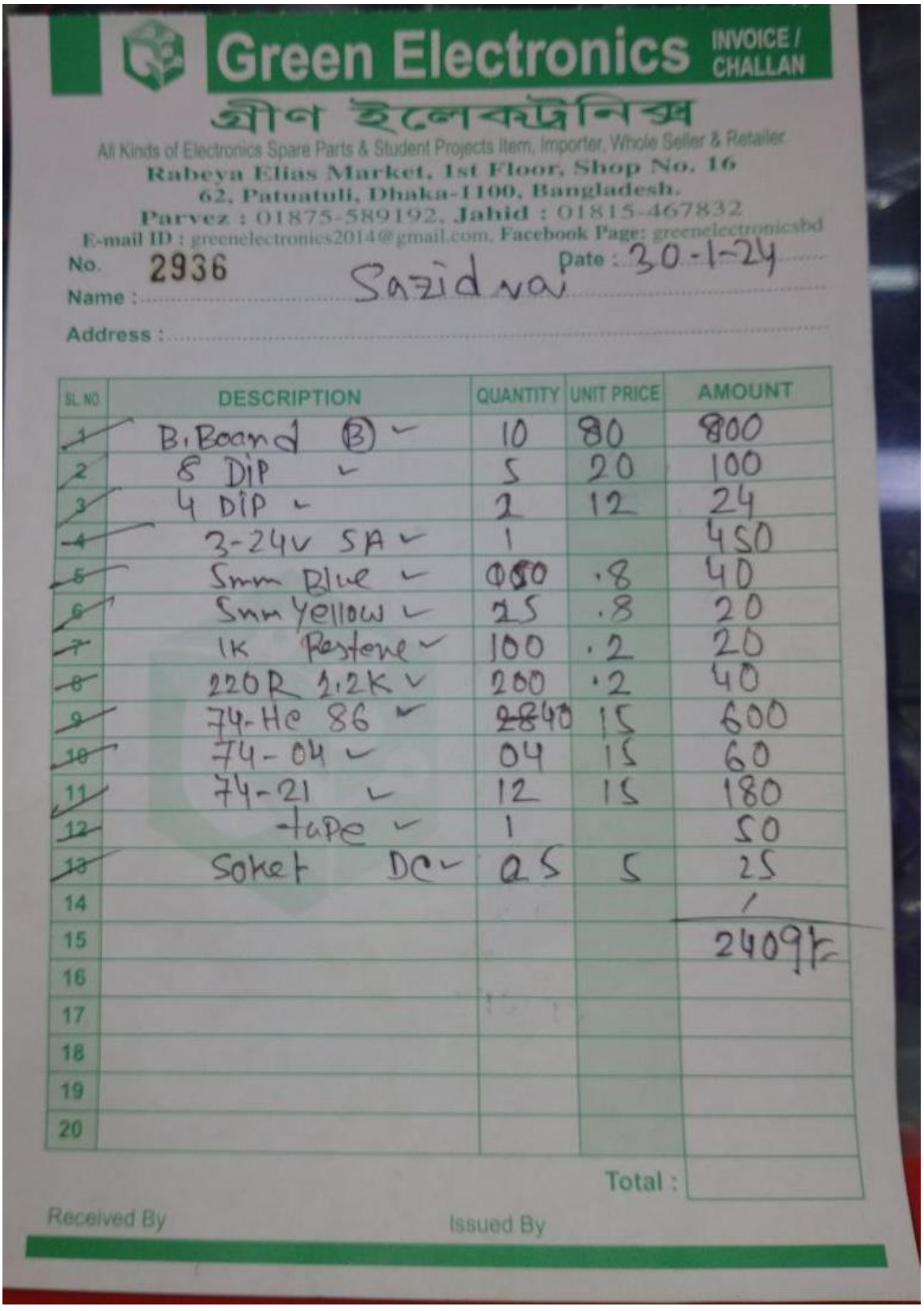
SL no.	Events	Date
1.	Submitting the project proposal for (12,8,4) Hamming code system	24 December 2023
2.	Modified project approved for (16,11,5) Hamming code system	29 December 2023
3.	Started planning to complete the project	28 January 2024
4.	Requesting for implementing (15,11,4) Hamming code system	28 January 2024
5.	Request approved	28 January 2024
6.	Started creating a schematic file on Proteus to simulate the entire project	29 January 2024
7.	Completing the schematic file on Proteus	30 January 2024
8.	Completed planning to implement the circuit in breadboard	30 January 2024
9.	Component purchasing for the first time	31 January 2024
10.	Trial on breadboard but we have failed	01 February 2024
11.	Started PCB designing	02 February 2024
12.	PCB Design completed	13 February 2024
13.	PCB ordered to TechshopBD	13 February 2024
14.	PCB received from TechshopBD by Red-Ex	16 February 2024
15.	Component purchasing for the second time	20 February 2024
16.	PCB soldering	22 February 2024
17.	Started working on hardware circuit with PCB	23 February 2024
18.	Hardware circuit completed	24 February 2024
19.	Testing and experiments	24 February 2024
20.	Photo or video capturing	24 February 2024
21.	Report writing	04 March 2024

Cost Analysis:

Used components with their price are given below.

SL no.	Component Name	Unit price (in Tk.)	Units used	Total Price (in Tk.)
1.	Bread-Board	80	6	480
2.	8 pin DIP switch	20	3	60
3.	4 pin DIP switch	12	1	12
4.	3-24V 5A DC adapter	450	1	450
5.	1k separated resistor	0.2	49	9.8
6.	9 pin SIP resistor (1k, A102J)	10	4	40
7.	X-OR IC (7486)	15	35	525
8.	NOT gate IC (7404)	15	1	15
9.	4 input AND gate IC (7421)	15	15	225
10.	DC adapter socket	1	5	5
11.	High rating white LED	1	49	49
12.	Male to Male connectors	60	6	360
13.	Male to Female connectors	60	2	120
14.	PCB printing	----	----	2750
15.	14 pin IC pad	4	51	204
16.	16 pin IC pad	3	5	15
17.	8 pin IC pad	1	5	5
18.	1 row, 40 pin female header	18	10	180
19.	PCB soldering	----	----	900
20.	Transportation Cost	----	----	720
Total cost				**7125**

Cash Memos:



Green Electronics INVOICE / CHALLAN

গ্রীণ ইলেকট্রনিক্স

All Kinds of Electronics Spare Parts & Student Projects Item, Importer, Whole Seller & Retailer

Rabeya Elias Market, 1st Floor, Shop No. 16

62, Patuatuli, Dhaka-1100, Bangladesh.

Parvez : 01875-589192, Jahid : 01815-467832

E-mail ID : greenelectronics2014@gmail.com, Facebook Page: greenelectronicsbd

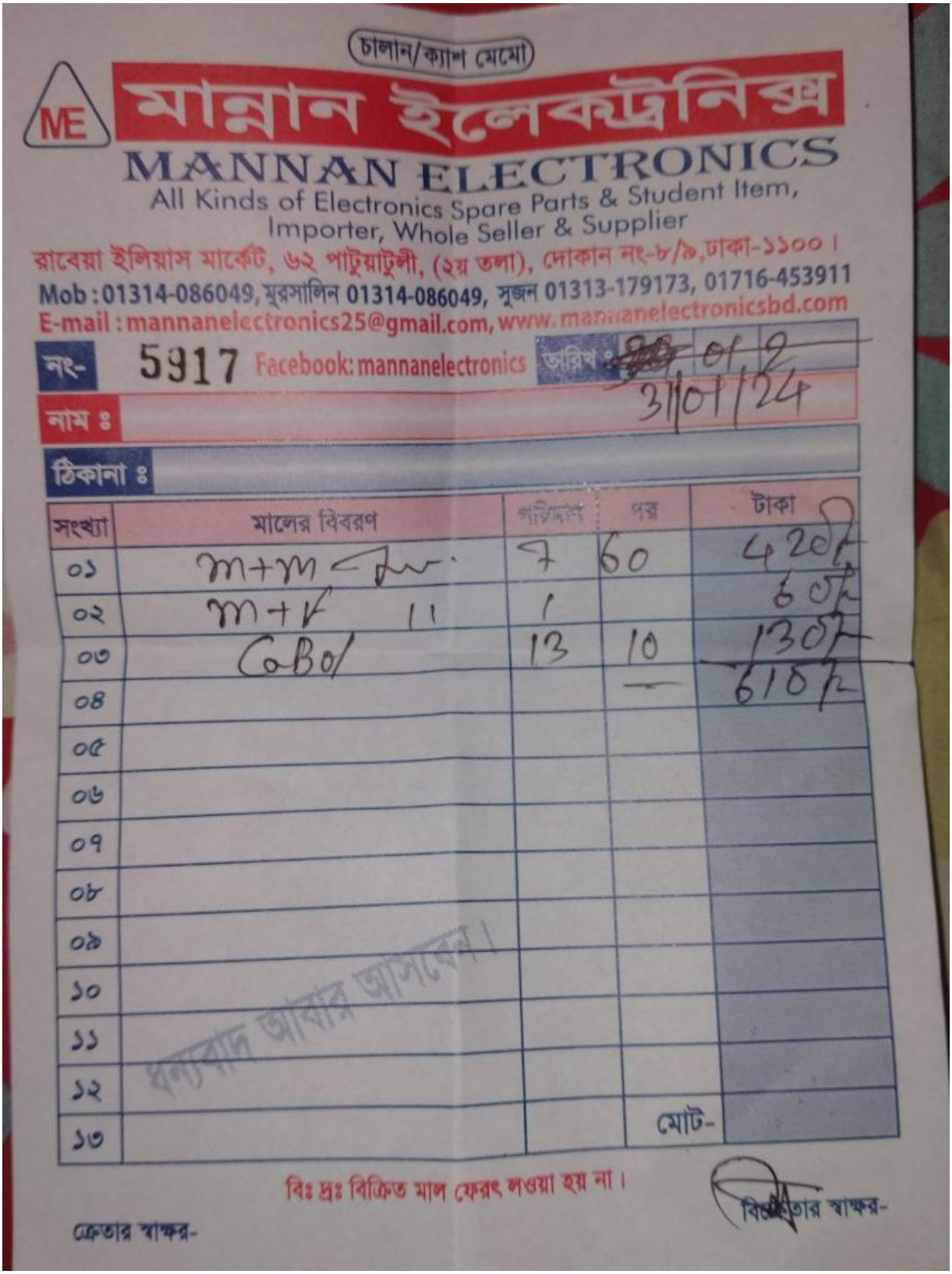
No. 2936 Date : 30-1-24

Name : Sazid vai

Address :

SL NO.	DESCRIPTION	QUANTITY	UNIT PRICE	AMOUNT
1	B. Board (B) ~	10	80	800
2	8 DIP ~	5	20	100
3	4 DIP ~	2	12	24
4	3-24V SA ~	1		450
5	5mm Blue ~	050	.8	40
6	5mm Yellow ~	25	.8	20
7	1K Resistor ~	100	.2	20
8	220R 1.2K ~	200	.2	40
9	74-HC 86 ~	2840	15	600
10	74-04 ~	04	15	60
11	74-21 ~	12	15	180
12	tape ~	1		50
13	Soket DC ~	05	5	25
14				/
15				2409 ~
16				
17				
18				
19				
20				
			Total :	

Received By Issued By



চালান/ক্যাশ মেমো

মান্নান ইলেকট্রনিক্স

MANNAN ELECTRONICS

All Kinds of Electronics Spare Parts & Student Item,
Importer, Whole Seller & Supplier

রাবেয়া ইলিয়াস মার্কেট, ৬২ পাটুয়াটুলী, (২য় তলা), দোকান নং-৮/৯,ঢাকা-১১০০।
Mob : 01314-086049, মুরসালিন 01314-086049, সুজন 01313-179173, 01716-453911
E-mail : mannanelectronics25@gmail.com, www.mannanelectronicsbd.com
Facebook: mannanelectronics

নং- 5917 তারিখ : 31/01/24

নাম ঃ

ঠিকানা ঃ

সংখ্যা	মালের বিবরণ	পরিমাণ	দর	টাকা
০১	m+m ~ Jw.	7	60	420/-
০২	m+F 11	1		60/-
০৩	CoBo/	13	10	130/-
০৪			—	610/-
০৫				
০৬				
০৭				
০৮				
০৯				
১০				
১১	ধন্যবাদ আবার আসবেন।			
১২				
১৩			মোট-	

বিঃ দ্রঃ বিক্রিত মাল ফেরৎ লওয়া হয় না।

ক্রেতার স্বাক্ষর-

বিক্রেতার স্বাক্ষর-

E-mail ID : greenelectronics2014@gmail.com, Facebook Page: greenelectronicbd

No. 3248

Date : 22/02/24.

Name : CASH

Address :

SL NO.	DESCRIPTION	QUANTITY	UNIT PRICE	AMOUNT
1	74 86	20	15	300
2	7404	4	15	60
3	74 21	16	15	240
4	33 Bt	1		900
5	Cutter	1		100
6	load Stan	1		300
7	Rang	1		190
8				1
9				20908
10	Lcd	80	1	80
11	Start	1		40
12				2210
13				
14				
15				
16				
17				
18				
19				
20				
			Total :	

Received By

Issued By

Future Works:

Here, we mention some future which we can be done on this project:

- We can add another parity bit in the 0th position of the 4x4 table to detect multiple errors though we will just be able to detect whether there is multiple error or not, but can't correct them.
- We may add some extra parity bits to detect and correct multiple errors in the codeword. But, for this we may have to change the error correcting procedure.

Application:

Applications of error correction with Hamming codes:

1. Data Storage
2. Computer Memory
3. Communication Protocols
4. QR Codes and Barcodes
5. Digital Television Broadcasting

Summary:

This project is about fixing mistakes in 15-bit digital data transmission using a special method called (15,11,4) Hamming Code. We first learned about how this code works and then put it into action using a computer program called Proteus. After making sure it works well in the virtual world, we turned it into a real circuit using a PCB design. The whole project cost us 7125 Taka, covering everything we needed. We tested the system thoroughly in both the computer simulation and the actual circuit. This project not only proves that the (15,11,4) Hamming Code is good at fixing mistakes but also shows it can be a practical and affordable solution for digital communication systems.

Conclusion:

In conclusion, our project focused on implementing the (15,11,4) Hamming Code to rectify errors in 15-bit digital data transmission. We gained a solid understanding of how the code operates, conducted thorough testing using the Proteus computer program, and translated our findings into a tangible circuit through PCB design. The real-world testing process allowed us to verify the effectiveness of the (15,11,4) Hamming Code in both simulated and practical environments. This comprehensive approach not only demonstrated the practical utility of the code but also highlighted its affordability, making it a promising and cost-effective solution for enhancing the reliability of digital communication systems. The successful integration of theory, simulation, and practical implementation assures the significance of our project in the realm of error correction methodologies.

References:

1. Modern Digital and Analog Communication Systems by B.P. Lathi, Zhi Ding (4th Edition)
2. https://www.youtube.com/@3blue1brown
3. https://www.researchgate.net/publication/259440422_A_Novel_Formulation_of_Hamming_Code
4. https://people.cs.umass.edu/~arya/papers/HammingERC.pdf
5. https://www.researchgate.net/publication/325083029_Bit_Error_Detection_and_Correction_with_Hamming_Code_Algorithm